



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Reconocimiento de Lenguaje
de Signos ASL mediante IA:
Implementación y
Comparativa de Rendimiento
en Raspberry Pi.**



Presentado por Hugo Gómez Martín
en Universidad de Burgos — 7 de julio de 2025

Tutor: Fernando Rivas Navazo

Tutor: Daniel Urda Muñoz



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Fernando Rivas Navazo, profesor del departamento de Ingeniería Electromecánica, área de Tecnología Electrónica y D. Daniel Urda Muñoz, profesor del departamento de Digitalización, área de Ciencia de la Computación e Inteligencia Artificial.

Expone:

Que el alumno D. Hugo Gómez Martín, con DNI 71567826Z, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado *Reconocimiento de Lenguaje de Signos ASL mediante IA: Implementación y Comparativa de Rendimiento en Raspberry Pi.*

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 7 de julio de 2025

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Rivas Navazo, Fernando

D. Urda Muñoz, Daniel

Resumen

Este proyecto presenta el desarrollo de un sistema de reconocimiento de lenguaje de signos Americano (ASL) que se basa en visión por computador y aprendizaje profundo, desplegado en una Raspberry Pi 5. Para llevarlo a cabo, combinamos la implementación de la detección de manos (**cvzone**) y la clasificación de gestos con una red neuronal convolucional (CNN) entrenada con **TensorFlow/Keras** sobre un dataset propio de 26 clases con 1000 imágenes cada una.

El modelo se prepara para su ejecución embebida con **TensorFlow Lite** y se integra en una aplicación *Streamlit* que permite tanto la clasificación en tiempo real, la clasificación con una imagen estática y además implementa una interfaz para poder re-entrenar el modelo con nuevos datos.

Descriptores

ASL, reconocimiento de gestos, Raspberry Pi, TensorFlow Lite, Keras, aplicación web, Redes Neuronales Convolucionales

Abstract

This project presents the development of an American Sign Language (ASL) recognition system based on computer vision and deep learning, deployed in a Raspberry Pi 5. To achieve this, we combine hand detection using `cvzone` and gesture classification with a convolutional neural network (CNN) trained with `TensorFlow/Keras` on a personal dataset of 26 classes with 1000 images each.

The model is prepared for embedded execution with `tensorflow Lite` and integrated into a `Streamlit` application that supports real-time classification, single-image classification, and provides an interface for retraining the model with new data.

Keywords

ASL, gesture recognition, Raspberry Pi, TensorFlow Lite, Keras, web application, convolutional neural networks

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vii
1. Introducción	1
1.1. Estructura de la memoria	3
1.2. Estructura de los anexos	3
1.3. Materiales adjuntos	4
2. Objetivos del proyecto	5
2.1. Objetivos generales	5
2.2. Objetivos técnicos	6
2.3. Objetivos personales	7
3. Conceptos teóricos	9
3.1. Lenguaje de signos ASL y su importancia	9
3.2. Reconocimiento de gestos y visión por computador	11
3.3. Redes neuronales convolucionales (CNN)	14
4. Técnicas y herramientas	19
4.1. Metodología de desarrollo y control de versiones	19
4.2. Entorno y librerías de desarrollo en Python	20
4.3. Herramientas de entrenamiento de modelos inicialmente utilizadas	31
4.4. <i>Hardware</i> empleado	33

5. Aspectos relevantes del desarrollo del proyecto	37
5.1. Fase de análisis y diseño	37
5.2. Construcción del <i>dataset</i>	39
5.3. Implementación y entrenamiento del modelo	42
5.4. Aplicación Streamlit (clasificador por imagen, tiempo real y módulo de entrenamiento)	48
5.5. Integración en Raspberry Pi 5	53
5.6. Incompatibilidad de mi red neuronal con <i>Pi AI Camera</i> . . .	54
5.7. Pruebas y comparativas	55
6. Trabajos relacionados	59
6.1. Reconocimiento de gestos de Google AI Edge	59
6.2. Sign Language Recognition with Convolutional Neural Networks	60
6.3. Google Teachable Machine	60
6.4. SignALL	60
7. Conclusiones y líneas de trabajo futuras	61
7.1. Conclusiones	61
7.2. Líneas de trabajo futuro	62
Bibliografía	65

Índice de figuras

1.1. Mapa de los países que adoptan ASL (o su variante) como lengua de señas oficial o cooficial. [15]	1
3.1. Abecedario ASL	10
3.2. Relación entre inteligencia artificial, aprendizaje automático y aprendizaje profundo. [28]	12
3.3. Marcadores de mano proporcionados por MediaPipe Hands . [1]	13
3.4. Mapa de características sobre una imagen aumentada de un perro. [21]	15
4.1. Logotipo de Pycharm [6]	21
4.2. Logotipo de Python 11 [12]	21
4.3. Logotipo de LaTeX [33]	22
4.4. Logotipo de Overleaf [38]	22
4.5. Logotipo de OpenCV [37]	23
4.6. Logotipo de TensorFlow y Keras [16]	24
4.7. Lototipo de TensorFlow Metal [41]	25
4.8. Comparativa con y sin aceleración gráfica	25
4.9. Logotipo de cvzone [17]	26
4.10. Logotipo de Streamlit [23]	27
4.11. Logotipo de Numpy [36]	28
4.12. Logotipo de Keras [14]	28
4.13. Logotipo de Matplotlib [35]	29
4.14. Logotipo de Scikit-learn [42]	29
4.15. Logotipo de psutil [29]	30
4.16. Logotipo de Teachable Machine [31]	32
4.17. Logotipo de ASUS ZenBook [40]	33
4.18. Logotipo de Mac Pro [34]	33
4.19. Logotipo de Raspberry Pi [39]	34

5.1. Imagen letra J extraida del conjunto de datos	39
5.2. Imagen letra Z extraida del conjunto de datos	40
5.3. Imagen de la arquitectura de la red neuronal	44
5.4. Logotipo del proyecto desarrollado	49
5.5. imagen del inicio del proyecto	50
5.6. Captura de la interfaz inicial de reconocimiento en tiempo real .	50
5.7. Captura de la interfaz de reconocimiento por foto	51
5.8. Captura de la interfaz de entrenamiento	52
5.9. Colores utilizados en el proyecto junto a su valor hexadecimal .	53
5.10. Comparativa de FPS realizada entre un portátil y la Raspberry Pi 5	55
5.11. Gráfica sobre la pérdida durante el entrenamiento del modelo .	56
5.12. Gráficas sobre la exactitud durante el entrenamiento del modelo	57
5.13. Matriz de confusion del modelo	58

Índice de tablas

1. Introducción

Más de 70 millones de personas con trastornos de audición utilizan diariamente alguna lengua de signos como su principal medio para poder comunicarse [24]. Aproximadamente medio millón de estadounidenses sordos o con dificultades en su audición utilizan ASL, mientras que aproximadamente solo hay un intérprete certificado disponible por cada 50 usuarios. El problema se agrava fuera de las grandes ciudades, con apenas 10000 interpretes titulados en toda Norteamérica, las zonas rurales son las que más dificultades tienen para acceder a estos servicios [5].

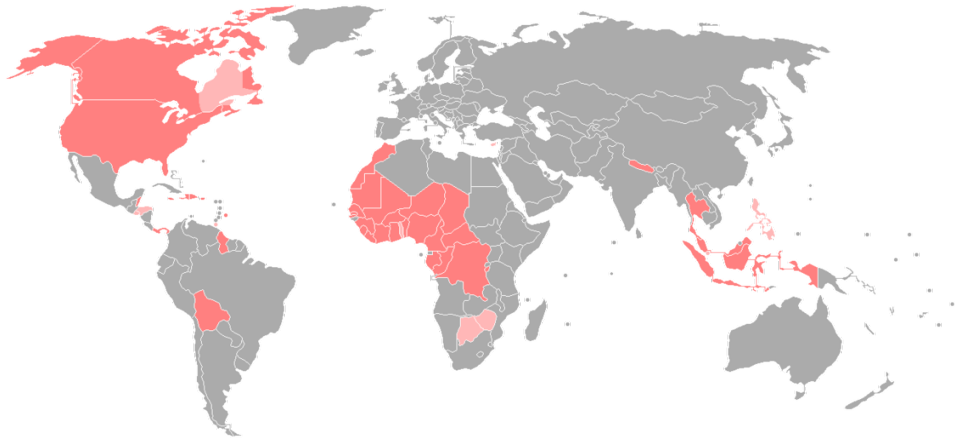


Figura 1.1: Mapa de los países que adoptan ASL (o su variante) como lengua de señas oficial o cooficial. [15]

Ante esta situación, decidí desarrollar un sistema automático que es capaz de traducir ese lenguaje de signos del alfabeto ASL en tiempo real a texto, reduciendo la dependencia de interpretes humanos y ampliarla a todos

los usuarios que padecen de esta discapacidad con una solución sencilla y eficaz en dispositivos de bajo coste actuales como la Raspberry Pi 5, modelo muy interesante que triplica la velocidad de procesamiento de su anterior versión[18].

Para poder llevarlo a cabo, procedemos a combinar la detección de manos en imágenes usando librerías como **cvzone** basadas en **MediaPipe** con la clasificación de gestos mediante una red neuronal convolucional (popularmente conocida como **CNN**) entrenada con **TensorFlow/Keras** sobre un conjunto de datos propio de 26 clases con 1000 imágenes cada una sumando un total de 26000 imágenes. Para la detección de la mano empleamos la librería **cvzone** basada en **MediaPipe** que nos permite localizar con precisión 21 puntos y aristas al rededor de nuestra mano [2], lo que facilita a nuestro modelo obtener la región de interés, mientras que el modelo **CNN** ofrece altas tasas de acierto en la clasificación de los respectivos gestos.

En esta documentación se detalla el desarrollo completo del proyecto, desde los objetivos planteados, los conceptos teóricos fundamentales, las herramientas y distintas técnicas empleadas, hasta los aspectos de implementación mas importantes y las pruebas de rendimiento y comparativas realizadas. El resultado es una aplicación desarrollada en **Streamlit** que permite:

- Detectar la mano del usuario a través de una cámara en tiempo real y a través de una *bounding box* o caja delimitadora que nos indica de izquierda a derecha la mano empleada, el porcentaje de acierto, y la letra reconocida por el clasificador. Además, se implementa métricas que cuantifican los fotogramas por segundo, los porcentajes de uso de CPU y RAM, así como la temperatura de la CPU.
- Clasificar gestos a partir de una imagen estática cargada por el usuario
- Re-entrenar el modelo con un nuevo conjunto de datos que el propio usuario proporcione, hasta 26 clases además de poder editar varios parámetros que considere. Después del entrenamiento, el usuario podrá descargar un archivo comprimido que contendrá dos modelos, uno de ellos optimizado y un fichero de texto con las diferentes clases que ha introducido para poder exportar su modelo.

De este modo se demuestra la viabilidad de este proyecto interactivo y portable para el reconocimiento del lenguaje de signos ASL.

1.1. Estructura de la memoria

La estructura de la memoria expuesta se compone de:

- **Introducción:** Breve descripción general del proyecto y del trabajo realizado junto a la estructura de la memoria y de los anexos.
- **Objetivos del proyecto:** Listado y descripción de los objetivos llevados a cabo a lo largo del proyecto.
- **Conceptos teóricos:** Descripción de los conceptos teóricos utilizados en el proyecto para su correcta comprensión y puesta en contexto.
- **Técnicas y herramientas:** Exposición de todas las herramientas y metodologías utilizadas para poder gestionar y construir satisfactoriamente el proyecto.
- **Aspectos relevantes del desarrollo del proyecto.** Se recogen los aspectos mas importantes que tuvieron lugar durante el desarrollo del proyecto y los diferentes retos que se han tenido que superar
- **Trabajos relacionados:** Exposición del estado del arte junto a un listado con diferentes aplicaciones que son similares al proyecto realizado con su respectiva descripción.
- **Conclusiones y líneas de trabajo futuras:** Diferentes conclusiones que hemos obtenido una vez finalizado el proyecto además de proporcionar diferentes mejoras e implementaciones que se pueden realizar en el futuro.

1.2. Estructura de los anexos

Además de la memoria, se proporcionan los siguientes anexos:

- **Plan de Proyecto Software:** Recoge la planificación del proyecto junto su respectivo análisis de viabilidad.
- **Especificación de Requisitos:** Se listan los requisitos definidos en la aplicación con respecto a los objetivos marcados.
- **Especificación de diseño:** Exposición del diseño de la aplicación además de proporcionar su arquitectura.
- **Manual del programador:** Los diferentes aspectos técnicos del proyecto y las distintas decisiones de implementación del código
- **Manual del usuario:** Una guía de usuario para poder manejar la aplicación y aprovechar todas las funcionalidades
- **Anexo de sostenibilización curricular:** Reflexión personal sobre los distintos aspectos de la sostenibilidad que se abordan en el proyecto.

1.3. Materiales adjuntos

Estos son los siguientes materiales que se adjuntan con la memoria:

- 3 memorias USB con todo lo necesario que requiere el proyecto
- Archivo ejecutable `.bat` que ejecuta la aplicación en el navegador (Microsoft Windows).
- Archivo ejecutable `.sh` que ejecuta la aplicación en el navegador (GNU/Linux).
- UN Archivo ejecutable `.exe` que ejecuta la aplicación en el navegador (Microsoft Windows) sin necesidad de instalar dependencias previas.
- *Dataset* tanto ordenado como desordenado con clases de la A a la Z con 1000 imágenes por cada clase.
- Varios modelos **Tensorflow/Keras** ya entrenados usados para la experimentación que pueden ser utilizados

Además, los siguiente recurso está accesible a través de internet:

- Repositorio del proyecto: [Repositorio GitHub del proyecto HandSign-DetectionProject](#).

2. Objetivos del proyecto

En este apartado se van a listar y describir de forma precisa cuales son los objetivos que se persiguen con la realización del proyecto, los vamos a dividir en tres tipos de objetivos, objetivos generales, técnicos y personales.

2.1. Objetivos generales

Estos tipos de objetivos son las metas principales que definen la razón de ser del proyecto y su resultado esperado desde un punto de vista funcional que han sido esenciales para guiar el desarrollo del proyecto:

- Desarrollar un sistema capaz de traducir cada letra del abecedario del lenguaje de signos ASL a texto, centrado en el alfabeto manual, que funcione de manera autónoma y ayude a disminuir la barrera de comunicación entre personas sordas o con trastornos de audición y oyentes.
- Demostrar la viabilidad de aplicar técnicas de visión por computador y *deep learning* además de poder optimizarlas en entornos embebidos para el reconocimiento de gestos, alcanzando un rendimiento suficiente para su uso interactivo.
- Comparar el rendimiento de distintas configuraciones y modelos dentro del proyecto, en particular evaluar mejoras obtenidas entre distintos modelos, además de evaluar el rendimiento con la respectiva optimización del modelo.

2.2. Objetivos técnicos

Estos objetivos a diferencia de los generales, son las metas específicas de desarrollo que se plantearon para poder lograr los objetivos generales que se citaron anteriormente. Incluyen hitos concretos, requisitos técnicos y criterios de éxito medibles:

- **Construir un conjunto de datos propio (*dataset*)** de imágenes de las 26 letras del alfabeto ASL, con un volumen suficiente de imágenes (26000 imágenes) y con la variabilidad necesaria (diferentes posiciones de la mano o también, gracias al *data augmentation*: iluminación, rotación, capas, etc.) para entrenar un modelo robusto. Se fijó como meta obtener al menos ~ 1000 imágenes por cada letra, mediante captura semiautomática.
- **Diseñar, implementar y entrenar con ayuda de una red neuronal convolucional (CNN)** capaz de clasificar las imágenes de 26 clases (de la A a la Z) con tasas altas de precisión en datos de validación.
- **Optimizar el proceso de entrenamiento** mediante validación y ajustes de hiperparámetros, añadiendo técnicas prácticas como *Early Stopping* (parada temprana) y *ReduceLROnPlateau* (ajuste adaptativo del *Learning Rate* o tasa de aprendizaje) para maximizar el entrenamiento del modelo. Adicionalmente, realizar una búsqueda de hiperparámetros mediante un *Random Search* para obtener los mejores y mejorar así el rendimiento del modelo base.
- **Construir un clasificador** donde podremos cargar nuestro modelo ya entrenado y poder así clasificar en tiempo real indicándonos en todo momento cada vez que reconozca la mano que letra reconoce con su respectiva *bounding box* o caja delimitadora que nos muestre que mano se esta reconociendo, la letra acertada y su confianza en porcentaje.
- Integrar el modelo entrenado en una **aplicación ligera con interfaz gráfica en una web** usando `Streamlit`, que permita al usuario interactuar fácilmente: visualizar en vivo la clasificación de lo que muestra la cámara (con indicación de la letra reconocida y su confianza en porcentaje), cargar imágenes individuales para clasificación y acceder a una funcionalidad extra de re-entreno donde el usuario final pueda añadir hasta 20 clases y su propio conjunto de datos, además de poder

editar varios hiperparámetros para que se ajuste completamente a sus necesidades.

- **Desplegar y ejecutar la aplicación en la Raspberry Pi 5** con el modelo infiriendo mediante **TensorFlow Lite**, verificando a través de las mediciones de *hardware* que el uso de CPU y memoria es manejable además de añadir un medidor de fotogramas por segundo para comprobar su rendimiento. También, en esta fase, se añadirían comparaciones de tiempos de inferencia del modelo en diferentes escenarios junto a comparaciones entre modelos.
- **Documentar** todos los procedimientos y distintos resultados obtenidos de forma rigurosa: mantener **control de versiones** de todo el proyecto utilizando **GitHub**, anotar la configuración de los distintos experimentos de entrenamiento realizados (arquitectura, hiperparámetros, tiempos de entrenamiento, métricas obtenidas), y preparar material visual de apoyo (gráficas de aprendizaje del modelo, ejemplos de predicciones, etc.) para el análisis de resultados

2.3. Objetivos personales

Además de los objetivos tanto técnicos como generales, una de las cosas mas importantes de este proyecto es el aprendizaje y el desarrollo personal para el estudiante, propios de un Trabajo Fin de Grado:

- **Ampliar conocimientos sobre el *deep learning*** y adentrarme al campo de la visión por computación pudiendo aplicar los conocimientos adquiridos de la asignatura *Artificial Intelligence and Knowledge Engineering* cursada durante mi estancia Erasmus+ en Breslavia, Polonia y profundizar en nuevas herramientas (**MediaPipe**, **TensorFlow/Keras**, etc.), adquiriendo experiencia en la construcción de un sistema completo de inteligencia artificial.
- Desarrollar habilidades de **ingeniería de software** planificando y organizando junto a mi tutor y co-tutor con cierta envergadura por falta de tiempo conviviendo con otras asignaturas, practicando metodologías de desarrollo (control de versiones, pruebas) y buenas prácticas de codificación en **Python 3**.
- Mejorar la **capacidad de investigación** y auto-aprendizaje sabiendo buscar bibliografía y proyectos relacionados además de aprender a

entender documentación técnica de nuevas librerías, experimentar con distintas soluciones y tomar decisiones fundamentales sobre qué dirección seguir en base a resultados.

- **Gestionar eficientemente el tiempo y los recursos** durante el desarrollo del proyecto estableciendo un calendario, fijar hitos y sobre todo, saber adaptarse a obstáculos que puedan ocurrir, redirigiendo el trabajo sin perder de vista los objetivos principales.
- Contribuir con una solución que tenga un **impacto social positivo**, es cierto que se trata de un prototipo académico, pero existe la motivación personal de que este proyecto sea un pequeño paso para que las tecnologías asistivas sean más comunes y accesibles para nuestra población.

Todos estos objetivos mencionados marcaron la dirección del proyecto. En los siguientes apartados se mostrará cómo se fueron abordando estos objetivos a lo largo del desarrollo y, en el apartado de conclusiones, se volverá sobre ellos para evaluar el logro de los mismos.

3. Conceptos teóricos

Para poder abordar mas adelante las técnicas y herramientas empleadas en el proyecto y respectivas decisiones, es necesario realizar previamente una explicación teórica sobre los conceptos mas relevantes.

Es este apartado se describe que es el lenguaje de signos ASL, por qué se ha elegido y su importancia. Después, se introduce el problema del reconocimiento de gestos por computador, diferenciando métodos mas primitivos y actuales; finalmente, se explican los fundamentos de las redes neuronales convolucionales (CNN), que es la pieza clave de nuestro proyecto junto a la explicación teórica de las distintas librerías implementadas que nos han ayudado a completar nuestra red y su respectiva representación.

3.1. Lenguaje de signos ASL y su importancia

El lenguaje de signos americano o también denominado *American Sign Language* (ASL) es una lengua de signos natural y completa usando el abecedario ingles americano con 26 letras de la A a la Z. Es el idioma principal de muchas personas sordas o con dificultades de audición en Norteamérica y también es utilizado por múltiples oyentes [25]. Aunque es cierto que nunca ha sido declarado idioma oficial de Estados Unidos, pero si tiene protección legal bajo la ley de Estadounidenses con Discapacidades (ADA) la cual impone a entidades la obligación de proveer interpretes acreditados u otros recursos para asegurar una comunicación accesible y fluida [32].

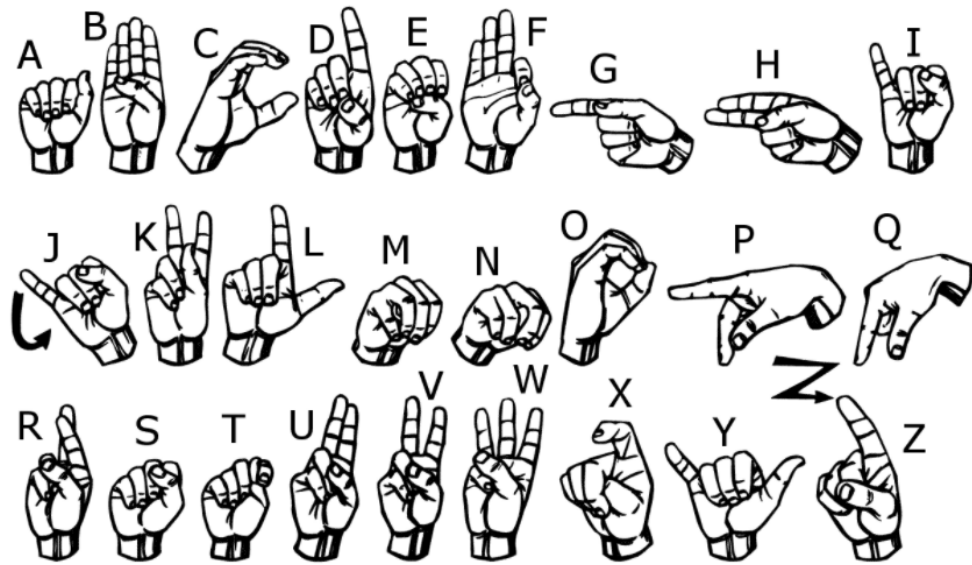


Figura 3.1: Abecedario ASL

Dentro del ASL además de gestos que pueden significar frases breves, también existe un objeto manual, el cual vamos a utilizar en el proyecto. Este alfabeto está compuesto por cada letra del alfabeto inglés y cada letra corresponde a un gesto único realizado con una sola mano [25]. Por ejemplo, para representar un nombre o palabras para las que no hay un gesto específico, se deletrea gesticulando con la mano.

Menos un par de letras que es cierto que implican movimiento como puede ser la "J", o la "Z", la mayoría de las letras se gesticulan con poses estáticas. Fue una característica fundamental para elegir este lenguaje en concreto entre otros, debido a que la mayoría de sus letras permanecen estáticas, menos ese par de letras para las cuales hemos encontrado la solución de mantenerlas estáticas en el final de su recorrido, así podemos realizar un conjunto de datos correcto y estático para poder clasificar correctamente.

En el contexto de este proyecto, centrarnos desde el inicio en el alfabeto ASL es de mucha utilidad, debido a que tenemos 26 clases diferentes que son relativamente fáciles de capturar en una cámara y su reconocimiento automático con el clasificador sienta las bases objetivas del proyecto.

3.2. Reconocimiento de gestos y visión por computador

Desde hace décadas, ha habido distintos enfoques a la hora de reconocer gestos de la mano u otros objetos. En términos generales, podemos distinguir entre enfoques clásicos basados en técnicas de visión artificial mas primitivos y tradicionales y otros enfoques actuales basados en el aprendizaje profundo o *deep learning*.

Técnicas clásicas

Antes de la aparición y del éxito del *deep learning*, el reconocimiento de gestos se lograba empleando algoritmos como descriptores locales. Un descriptor local que se solía utilizar anteriormente era el uso de descriptores locales como HOG (*Histogram of Oriented Gradients*) para captar características de la imagen como por ejemplo la textura del objeto, su respectiva orientación, etc. Mas adelante, estas características se clasificaban con un clasificador tradicional como podrían ser un SVM o un HOG-MO entrenado previamente para distinguir entre distintas letras o gestos [19].

Otras técnicas también podrían ser convertir la imagen al espacio de color HSV con YCrCb pudiendo así aislar la región de piel del fondo usando rangos de colores adecuados. Una vez aislada la mano, se extraen características geométricas de su contorno y postura debido a que la forma del perímetro de la mano, el perfil de como esta orientada la palma y la posición de los dedos, como en el caso de nuestro proyecto, ofrecen información muy valiosa [8].

Técnicas actuales basadas en aprendizaje profundo aplicado a imágenes

Antes de abordar este apartado, es fundamental saber comprender y diferenciar entre *machine learning* y *deep learning*. Ambas forman parte de la inteligencia artificial y a su vez, el *deep learning* forma parte de *machine learning*, si están separadas, ¿En que se diferencian exactamente?.

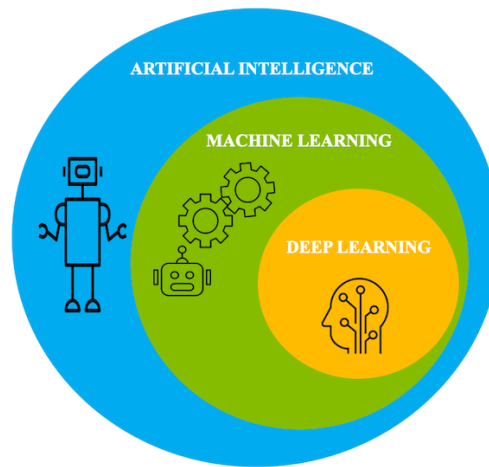


Figura 3.2: Relación entre inteligencia artificial, aprendizaje automático y aprendizaje profundo. [28]

Para poder comprenderlo mejor, voy a explicarlo con un ejemplo práctico y general sobre reconocimiento de imágenes:

Empecemos por el *machine learning*, digamos que le ofrecemos a un clasificador fotos de un millón de gatos y fotos de un millón de perros junto a sus respectivas etiquetas, a medida que va siendo entrenado con todas estas imágenes, va a ir poco a poco reconociendo patrones, por ejemplo, detecta que los gatos tienen las orejas en punta y rígidas, también reconoce otros patrones como los ojos, al igual que con los perros, también aprende patrones como que tienen por lo general las orejas caídas, no tienen bigotes. Por lo tanto, gracias a este ejemplo que acabo de presentar podemos concretar que el *machine learning* va poco a poco reconociendo patrones y poco a poco van clasificando las imágenes.

El *deep learning* es un tipo de *machine learning* que es multicapa, es decir, continuando con el ejemplo anterior, tenemos varias capas, una capa inicial una más profunda, otra más profunda y así sucesivamente, donde cada capa va reconociendo detalles, por ejemplo, una capa se fija en la silueta del gato, otra capa en las orejas, otra capa en el pelaje, la cola y así sucesivamente.

Gracias a este ejemplo podemos concretar que gracias a la estructura multicapa, los modelos de *deep learning* son capaces de aprender representaciones más abstractas y sobre todo más precisas, mientras que los métodos de *machine learning* dependen en mayor medida de las característi-

3.2. RECONOCIMIENTO DE GESTOS Y VISIÓN POR COMPUTADOR

cas manualmente diseñadas y extraídas. una vez comprendida la diferencia, proseguimos con el apartado.

En estos años, el éxito del *deep learning* ha transformado totalmente el campo de reconocimiento de gestos. Actualmente, se utilizan redes neuronales convolucionales (CNN) entrenadas con grandes conjuntos de datos diferenciados entre clases distintas entre sí. A diferencia de los métodos clásicos, los modelos CNN son capaces de aprender automáticamente características importantes y diferenciales de las imágenes de ese conjunto de datos (forma de los dedos, perímetro, texturas, sombras, etc.) durante el entrenamiento. Esto ha permitido que la precisión sea mucho mas alta en lugar de aprender de descriptores predefinidos debido a que estos modelos CNN son mucho mas robustos a variaciones de iluminación o fondos difíciles de diferenciar.

Es cierto que en este proyecto hemos ayudado al clasificador con la librería *cvzone* basada en *MediaPipe Hands* de Google capaz de detectar 21 puntos de los nudillos y 21 aristas entre las distintas falanges de la mano y la propia palma en 3D de una mano a partir de una sola imagen en tiempo real y hasta múltiples manos (para alcanzar el objetivo de nuestro proyecto, solo se ha habilitado una mano a la vez)[2].

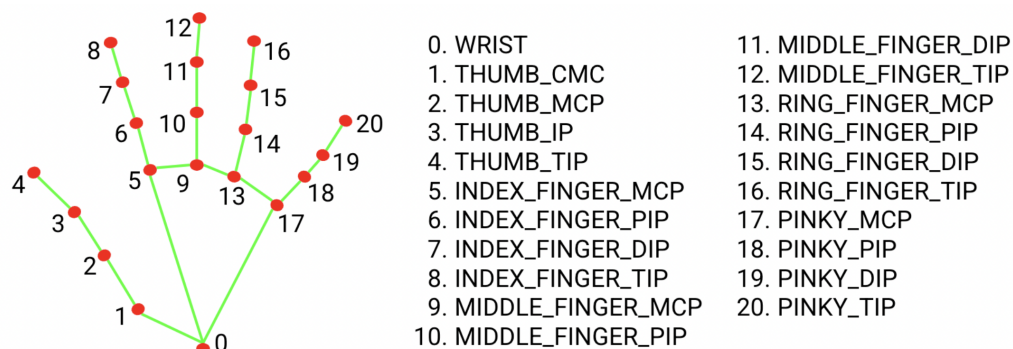


Figura 3.3: Marcadores de mano proporcionados por *MediaPipe Hands*. [1]

Podemos destacar que nuestro modelo diferencia hora mucho mejor las 26 clases debido a que tiene un patrón mucho mas claro ahora de nuestra mano, por lo tanto, si introducimos un conjunto de datos previamente procesados con *cvzone* y además, añadimos esta librería en nuestro clasificador, el modelo va asociar con mas confianza el gesto que detecta en tiempo real con una de las clases del clasificador, por eso a la hora de realizar la inferencia en tiempo real, muestra un porcentaje tan alto de confianza que, habiendo obviado esta librería, no hubiésemos logrado. Podríamos decir que combinamos lo

mejor de los dos mundos, aprovechamos un modelo pre-entrenado que nos otorga la librería *cvzone* para entender la mano (reduciendo la dependencia de otras condiciones externas) y encarga nuestra red principal la tarea de reconocer únicamente el signo.

Podemos concretar que el paradigma actual ha cambiado, los algoritmos basados en aprendizaje profundo dominan el reconocimiento de lenguaje de signos, debido a que son mucho mas precisos y aunque son un poco mas complejos, tienen flexibilidad para aprender e datos, sustituyendo a las técnicas primitivas que vimos anteriormente basadas en reglas fijas.

3.3. Redes neuronales convolucionales (CNN)

Las redes neuronales convolucionales, mayormente conocidas como CNN (*Convolutional Neural Networks*), se han adentrado con paso firme en el campo de la inteligencia artificial, mas concretamente en el procesamiento de imágenes por computador. Su capacidad para reconocer patrones visuales y reconocer imágenes con una precisión sorprendente ha permitido avances asombrosos en áreas como la visión artificial, el reconocimiento facial, o en otros campos externos como pueden ser la medicina o la conducción autónoma [11].

Para poder entender esto claramente tenemos que analizar antes los principios generales que hay dentro del proceso de visión, no el artificial, si no el humano. Volvamos al ejemplo de los perros y los gatos, cuando vemos a un perro por la calle, fácilmente sabemos que es lo que estamos viendo, un animal con cuatro patas, orejas caídas, que tiene ojos, hocico, boca y su cuerpo además de ser alargado tiene pelaje y cola, Esto es porque tú seguramente has escaneado la imagen que te proporcionan tus ojos y has detectado esta presencia de elementos que te dan como resultado, gracias a tu aprendizaje, la presencia de un perro. Pero, ¿Por qué sabemos que hemos detectado estos elementos que son comunes a un perro, por ejemplo, el hocico? Porque hemos detectado una serie de características que conforman un hocico, por ejemplo que suele variar entre negra y marrón, tiene forma ovalada con textura rugosa y dos orificios a los extremos del óvalo que conforma su nariz, y esto lo hemos sabido reconocer porque previamente sabemos reconocer patrones geométricos como círculos, óvalos, cambios de contrastes, tonalidades, texturas, etc. Por lo tanto, si lo analizamos dentro de nuestra cabeza, descubrimos que nuestro cerebro realiza un procesamiento en cascada donde primero se analizan esos elementos básicos y generales donde

en capas posteriores estos elementos sencillos se combinan para identificar patrones bastante mas complejos e identificar con confianza los elementos que nos rodean.

Esta breve ejemplificación es fundamental para poder entender que este tipo de red es el núcleo de la solución de imágenes propuesta. Una red CNN es un tipo de red neuronal que esta diseñada para poder procesar datos con estructura espacial. Esta estructura espacial se puede reconocer como una matriz de pixeles, su resolución nos muestra la cantidad de píxeles que hay en la imagen, por lo tanto, para poder adentrarnos a esta explicación, vamos a observar esta imagen como una matriz (que en realidad esta compuesta por tres matrices debido a que esta compuesta por los tres canales de color en nuestra red (R(Red/Rojo) G(Green/Verde) B(Blue/Azul))).

A diferencia de una red neuronal densa tradicional, donde cada neurona de una capa están conectadas a todas las otras neuronas de la capa anterior, en un CNN las primeras capas de una CNN aplican filtros convolucionales también llamados *kernels* que actúan como detectores de patrones. Esos diferentes patrones se combinan en capas mas profundas para detectar características mucho mas complejas como bordes, texturas y formas como pudimos ver en el ejemplo anterior y así sucesivamente realizando y construyendo una jerarquía de características.

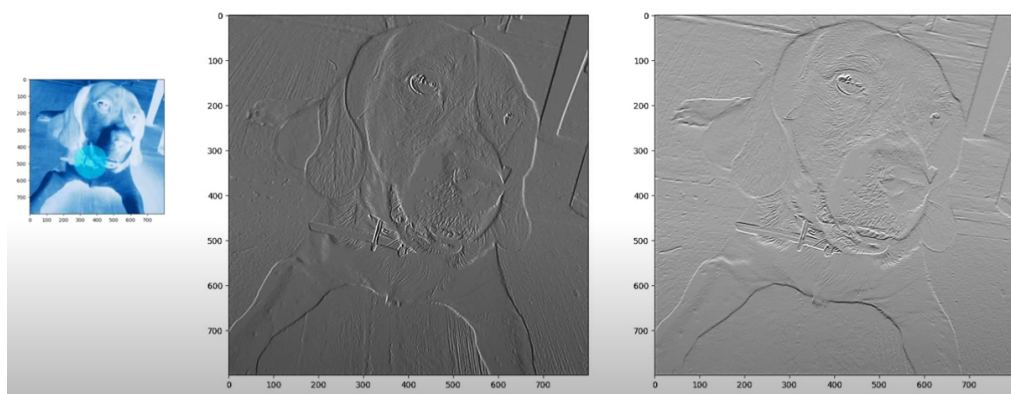


Figura 3.4: Mapa de características sobre una imagen aumentada de un perro. [21]

En nuestra red, este modelo puede aprender a diferenciar, por ejemplo, una situación en la que únicamente los dedos índice y corazón están levantados, el modelo puede clasificarlo porque gracias al aprendizaje proporcionado por las capas convolucionales, identifica dicho gesto y lo asocia, si es claro, correctamente con la etiqueta "V". Esto es algo difícil de programar si lo

llegamos a realizar con reglas programadas, peor la CNN aprende gracias a todos estos conjuntos de datos anotados con etiquetas (*labels*).

Un modelo típico para clasificación de imágenes incluye varias convolucionales con capas de *pooling* (submuestreo), las cuales reducen su resolución de la representación para poder así reducir su dimensionalidad debido a que realmente no necesitamos una imagen tan detallada para realizar la tarea de clasificación. Por ejemplo, no queremos que nuestra red se fije en la textura de la pared o de las manos para poder determinar que un gesto es la letra A, o la letra B. Además estamos optimizando el modelo debido a que estamos reduciendo la dimensionalidad de los datos acelerando así el entrenamiento de nuestra red neuronal. Finalmente, una o mas capas completamente conectadas (densas) toman todas las características extraídas y muestran la decisión de la clasificación (mediante activaciones *softmax* para poder obtener así las probabilidades por clase). Durante el entrenamiento, el modelo ajusta sus *kernels* o filtros y pesos internos minimizando una función de pérdida (el clasificación, entropía cruzada o *crossentropy*) usando gradiente descendente y *backpropagation*. el resultado es un conjunto de filtros que detectan gestos específicos del *dataset*.

En nuestro proyecto inicialmente habíamos diseñado una arquitectura CNN a medida para encajar con nuestro *dataset*. El modelo constaba de 4 bloques de convolución + *pooling* + normalización que iban aumentando el numero de filtros (profundidad) progresivamente como si fuese un embudo, con una capa densa intermedia y una capa de salida de 26 neuronas (una por cada letra). Se realizó un **random search** de 54 *trials*. Además, aplicamos *Batch Normalization* en las capas convolucionales (para acelerar e intentar estabilizar el entrenamiento) y *dropout* en la capa densa (para poder evitar el sobreajuste). En fase de entrenamiento, también se incorporó *Early Stopping* para detener el entrenamiento cuando la precisión de la validación deja de mejorar. Pero es cierto que aún teniendo esta estructura compleja, producía mucho desajuste y mostraba una matriz de confusión desbalanceada con muy baja densidad en la diagonal.

Esta red neuronal se sustituyó por un modelo preentrenado para entrenar nuestro *dataset* sobre el. Específicamente es un **EfficientNetB0** preentrenado en **Imagenet**, al que añadimos una cabecera ligera para adaptar el modelo a ASL de 26 clase (**GlobalAveragePooling2D**, **BatchNormalization**, **Dropout**). Este enfoque de transfer learning nos permite aprovechar las características ya aprendidas en millones de imágenes, reduciendo drásticamente el tiempo de entrenamiento y la necesidad de datos masivos en nuestro *dataset*. Además, aplicamos **Data Augmentation** para añadir va-

riedad a los datos junto a un `EarlyStopping` y `ReduceLROnPlateau`. Este modelo finalmente converge mejor con muy poco sobreajuste y una diagonal muy marcada indicándonos que la red clasifica realmente bien.

En definitiva, las CNN nos proporcionan ese "motor de reconocimiento" del sistema. una vez entrenada, se realiza la inferencia que es el proceso de tomar nuestro modelo CNN previamente entrenado, mostrarle datos nuevos para hacer que el propio modelo realice predicciones [10]. En milisegundos produce una predicción de qué letra representa ese gesto que se presenta ante la cámara. La precisión alcanzada por nuestra red neuronal convolucional en clasificación de gestos del alfabeto ASL es muy alta, y como veremos en próximos apartados más detalladamente, nuestro modelo logró aprender de manera bastante fiable diferencias entre estas 26 clases. Esta información valida el uso de las CNN como un gran modelo para el reconocimiento de lenguaje de signos ASL en imágenes.

4. Técnicas y herramientas

En este capítulo principalmente se describen las principales técnicas metodológicas y las herramientas empleadas para llevar a cabo el proyecto. Esto abarca desde la forma en que se organizó el desarrollo, como la metodología o el control de versiones hasta el entorno de programación utilizado (lenguaje, IDE, librerías, etc.) y el **hardware** disponible. También se comentarán las alternativas consideradas en cada caso además de justificar las elecciones tomadas a lo largo del desarrollo.

4.1. Metodología de desarrollo y control de versiones

Se intentó seguir en todo momento una **metodología ágil** adaptada al horario, donde la mayoría de *sprints* se componían de al menos dos semanas. Inicialmente es cierto que se hicieron varios prototipos rápidos para comprobar la viabilidad (por ejemplo capturar imágenes de la cámara desde la librería **OpenCV**, entrenar un modelo con hiperparámetros poco ajustados con pocas capas ocultas, realizar un programa clasificador no funcional cuyos hiperparámetros no coincidían con el modelo entrenado, etc.) como se realiza en cualquier proyecto. A partir de ahí, se fueron arreglando detalles del código como por ejemplo implementar funcionalidades nuevas mejorando partes del código como el cargado de imágenes desde el dispositivo en el entrenamiento del modelo por un código mucho mas simple, intuitivo y funcional. Mas adelante se verifico que un modelo simple podía aprender algo; después de comprobar que sí, se mejoró y mas tarde se cambió la arquitectura además de los hiperparámetros con un **random search** que nos otorgó buenos resultados en nuestra red. Por último, esta se integró parte

de la interfaz **Streamlit** además de proceder a la optimización del modelo para la Raspberry Pi y micro-ordenadores o ordenadores de bajos recursos en general.

Este enfoque permitió flexibilidad para reorganizar tareas según los distintos obstáculos encontrados (por ejemplo, tras descartar el uso de la cámara con NPU, reorientaron los esfuerzos a optimizar el modelo en CPU)

Flujo iterativo-exploratorio en una única rama *main*

Para el control de versiones del código se utilizó Git, alojando el repositorio inicialmente privado en GitHub (en las versiones finales se hizo público). Se trabajó en una **única rama principal main**, dado que el equipo está compuesto solo por una persona y el proyecto no requirió a mi parecer ramificaciones complejas. Aún así, Git fue fundamental para llevar registro de los cambios, poder volver a diferentes versiones estables en caso de error o poder sincronizar el proyecto entre ordenadores para poder realizar correctamente los entrenamientos en local, esto fue fundamental para poder entrenar el último modelo que había desarrollado principalmente en mi ordenador personal con una GPU integrada y poder transportarlo así a un ordenador con una mejor tarjeta gráfica en este caso dedicada para acelerar el entrenamiento utilizando aceleración por GPU.

Aunque no hubo colaboradores, la práctica de usar una herramienta de control de versiones y utilizar una metodología contribuyó a mantener un código y un proyecto **bien estructurado**, previendo posibles mantenimientos futuros. En cuanto a gestión de proyectos, se emplearon *to-do-lists* para priorizar funcionalidades del *sprint* y se mantuvo una comunicación frecuente con los tutores para revisar el estado de mi proyecto y diferentes direcciones para alcanzar los objetivos.

4.2. Entorno y librerías de desarrollo en Python

Entorno de desarrollo

En este apartado se describirán las bases para realizar el proyecto, además de las distintas herramientas para el desarrollo de la documentación.

Pycharm community Edition (IDE)



Figura 4.1: Logotipo de Pycharm [6]

El proyecto se desarrolló íntegramente en el entorno de desarrollo integrado (IDE): **Pycharm Community Edition 2024.1.4**. Dicho IDE genera automáticamente el entorno virtual `.venv`, lo que posibilitó poder instalar de forma asilada todas las dependencias del proyecto, evitando así conflictos de versiones con otros proyectos presentes en el equipo. Se usó la versión *Community* de **PyCharm** por que resultó imposible acceder a la versión *Professional* con las claves facilitadas por la Universidad de Burgos.

Python 3.11



Figura 4.2: Logotipo de Python 11 [12]

Para la ejecución del código se empleó: Python 3.11.7, lanzada en 4 de diciembre del 2023. No se eligió por ningún motivo en particular, simplemente ya se hallaba instalada en el equipo debido a la realización de proyectos con el mismo IDE posteriores a la realización de este trabajo. Al ser una versión relativamente reciente además de proporcionar compatibilidad total con las bibliotecas requeridas, no se consideró necesario actualizarla.

L^AT_EX



Figura 4.3: Logotipo de LaTeX [33]

L^AT_EX es un sistema que se centra en la composición de documentos de texto, especialmente diseñado para documentos científicos o de investigación (principalmente usado en documentos relacionados con la ingeniería o las matemáticas). Maneja muy bien formulas matemáticas, referencias cruzadas, referencias bibliográficas y figuras, proporcionando resultados de alta calidad tipográfica sin preocuparnos de los detalles de diseño.

Overleaf



Figura 4.4: Logotipo de Overleaf [38]

Overleaf es un editor en línea para proyectos L^AT_EX en la nube. Es una plataforma donde podemos crear documentos y además compilarlos en el propio editor. ofrece colaboración con mas miembros para trabajar en equipo o en grupos. Está orientado principalmente a aquellas personas que no quieran pelearse con instalaciones locales para poder trabajar con este tipo de documentos o prefiera trabajar en equipo.

Librerías utilizadas para el desarrollo

A continuación, voy a mostrar las librerías utilizadas más destacadas con una breve descripción sobre ellas

Se han necesitado todas las librerías expuestas en la tabla para poder alcanzar los objetivos, no obstante hay tres librerías fundamentales que necesitan ser explicadas con más detalle:

OpenCV

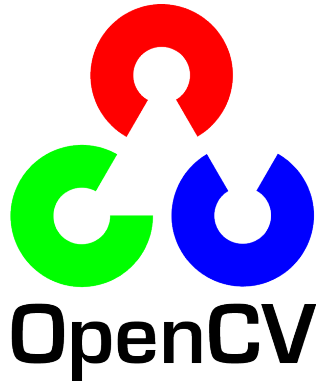


Figura 4.5: Logotipo de OpenCV [37]

Es la biblioteca de visión por computador de código abierto mas extendida y popular [26]. Ofrece cientos de algoritmos sobre visión por computador ademas de funciones para cargar, procesar y analizar imágenes y video en tiempo real [27]. En este proyecto al cargar `OpenCV` con `cv2` se emplea para tarea como la capturas de las imágenes de la cámara, preprocesamiento como redimensionar, recortar el área de la mano y su respectivo dibujo junto a las anotaciones en pantalla como por ejemplo dibujar el *bounding box* o el resultado de la predicción y gestión general del flujo de video. Es una herramienta optimizada y escrita en C++, disponible para Python [26][27]. Su eficiencia es fundamental para lograr que el sistema funcione, no solo fluido en nuestro equipo, si no en un *hardware* limitado como la Raspberry Pi.

TensorFlow y Keras



Figura 4.6: Logotipo de TensorFlow y Keras [16]

Son las librerías principales que construyen, entrenan y ejecutan el modelo CNN que realiza la clasificación de nuestro alfabeto ASL. **TensorFlow** es un *framework* de **deep learning open-source** creado por Google utilizado con el objetivo de construir y entrenar redes neuronales. En la versión que estamos utilizando en nuestro proyecto que es la 2.12 integra de forma integral **Keras** como su API de alto nivel para crear y entrenar modelos de aprendizaje profundo [30]. Esto quiere decir que **Keras** funciona como una interfaz simplificada dentro de **TensorFlow**, con el objetivo de crear prototipos rápidamente tanto en campos como la investigación e ingeniería o entornos orientados a la enseñanza [30].

En este proyecto se ha utilizado **Tensorflow** (con la API de **Keras**) para entrenar y diseñar una red neuronal convolucional capaz de reconocer el lenguaje de signos ASL, aprovechando la facilidad de **Keras** para definir arquitecturas CNN sin complicar ni aumentar el número de líneas de nuestro código siendo así más fácil gestionar el ciclo de entrenamiento y su mantenibilidad.

Además, para aprovechar la GPU AMD Radeon D700 del Mac Pro 6.1 durante el entrenamiento local, se integró la extensión **TensorFlow Metal**.



Figura 4.7: Lototipo de TensorFlow Metal [41]

Esta extensión de **TensorFlow** permite usar la API gráfica **Metal** de Apple como backend de **Tensorflow**. Gracias a ello, las operaciones de tensores se aceleran notablemente sin necesidad de *hardware* NVIDIA ni CUDA, reduciendo así los tiempos de nuestro entrenamiento. En esta gráfica podemos observar los tiempos que marcan nuestras épocas con ella y sin ella aplicada:

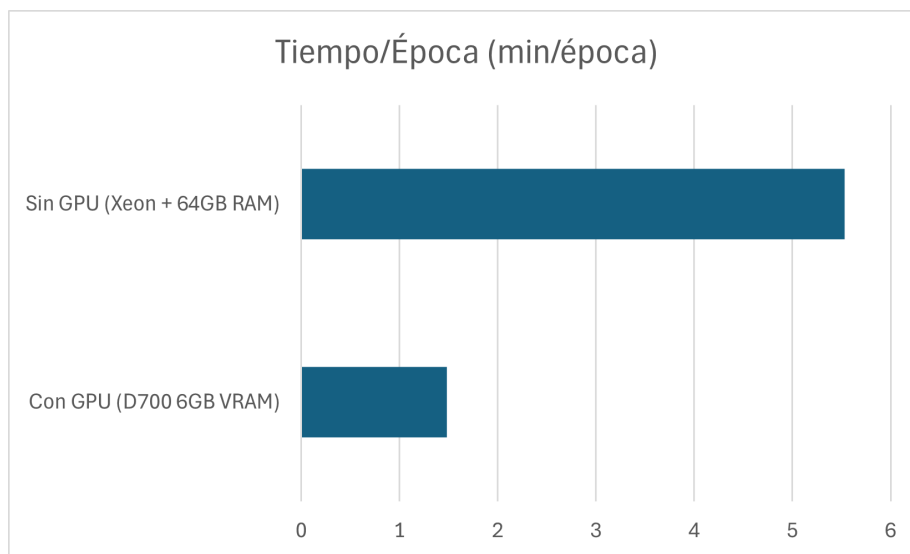


Figura 4.8: Comparativa con y sin aceleración gráfica

cvzone y **MediaPipe**



Figura 4.9: Logotipo de cvzone [17]

Es una librería que facilita el reconocimiento de nuestra mano. En concreto, **cvzone** provee funcionalidades listas para usar, como su módulo de seguimiento de manos (**HandTrackingModule**) que internamente usa **MediaPipe Hands** pero expone una interfaz mas sencilla para obtener información como la ubicación de la mano, posición de los dedos, dibujo de los puntos donde se ubican los nudillos, etc. En este proyecto, **cvzone** simplifica el código necesario para detectar la mano, por ejemplo, con unas pocas líneas se inicializa un *HandDetector* propio de esta librería y se obtiene un *bounding box* de la mano presente en la imagen. Por debajo, se usa **OpenCV** para el manejo de imagen y las funciones de **MediaPipe**, pero nos ayuda a ahorrarnos integrar estas librerías manualmente en nuestro código para ubicar visualmente nuestra mano. En resumen, esta librería nos ayuda a acelerar el desarrollo de nuestro proyecto pudiendo importar su modulo **HandTrackingModule**. Esta librería es mantenida por la comunidad de *Computer Vision Zone* y resulta muy útil por si no queremos complicar nuestro código y dejarlo limpio importando su módulo. Además, no solo cuenta con **HandTrackingModule**, si no también otros módulos como detección de rostros, seguimiento de objetos, reconocimiento de poses humanas, etc. todo integrado con módulos de forma coherente e intuitiva. Vale destacar que **cvzone** no reemplaza a **OpenCV** o **MediaPipe**, sino que se construye sobre ellas (de hecho, utiliza estas dos librerías internamente), por lo que usa la robustez y complejidad de esas librerías a la vez que ofrece una sintaxis mucho mas amigable.

Streamlit



Figura 4.10: Logotipo de Streamlit [23]

Streamlit es un Framework web que permite crear aplicaciones web interactivas completamente codificadas en Python (al igual que todo el ecosistema del proyecto) de forma rápida y sencilla, enfocadas sobre todo a proyectos de ciencia de datos y *machine learning*. Esta biblioteca facilita a desarrolladores sin experiencia en front-end la construcción de interfaces de usuario atractivas con pocas líneas de código [23]. Esa es una de las principales razones por las cuales se ha escogido **Streamlit** para la realización de una web accesible e interactiva escondiendo la complejidad del proyecto. Por medio de esta librería, se pudo desplegar una interfaz donde se ejecuta y visualiza tanto el reconocimiento en tiempo real como el reconocimiento de variables estáticas, además de incluir un apartado muy parecido a **Teachable Machine** de Google donde el usuario puede entrenar el modelo con su propio conjunto de datos además de poder editar diferentes hiperparámetros como la cantidad de épocas, tamaño del *batch* y *learning rate*.

Además de la librería **Streamlit**, también hemos utilizado un componente externo (realizados por la comunidad):

- **streamlit-option-menu**: Permite mostrar un menú de opciones interactivo en la interfaz. En concreto (en nuestra interfaz), proporciona un menú lateral con elementos clicables para navegar entre diferentes interfaces de la aplicación.

NumPy



Figura 4.11: Logotipo de Numpy [36]

Es una librería fundamental en Python que proporciona *arrays* multidimensionales. Es ampliamente utilizada en ciencia de datos, *machine learning* y visión por computador por su eficiencia a la hora de manejar grandes cantidades de datos. En este proyecto se emplea para poder procesar las imágenes.

Keras Tuner



Figura 4.12: Logotipo de Keras [14]

librería que tiene el objetivo de simplificar en gran medida la tarea de optimización de hiperparámetros en modelos de *deep learning*. **Keras Tuner** principalmente permite definir rangos de búsqueda para hiperparámetros y probar automáticamente tantas veces como intentos (*trials*) queramos sin tener que realizar la prueba de hiperparámetros manualmente uno a uno. Clave en nuestro proyecto a la hora de hacer la búsqueda profunda por rangos (*random search*).

Matplotlib



Figura 4.13: Logotipo de Matplotlib [35]

librería especializada en la creación de gráficos y visualizaciones de dos dimensiones. Con esta librería se pueden realizar múltiples tipos de gráficos (líneas, barras, histogramas..). En nuestro proyecto se utiliza para la comparación gráfica entre modelos obteniendo dos gráficas de pérdida y precisión por cada modelo entrenado, permitiendo visualizar así fenómenos de sobreajuste.

Scikit-learn



Figura 4.14: Logotipo de Scikit-learn [42]

Esta librería la usamos por su función `confusion matrix` la cual nos ayuda en nuestro proyecto a crear y así poder visualizar la matriz de confusión para ver los resultados de nuestro modelo ya entrenado. Con su función `display` podemos visualizar esa matriz de forma gráfica coloreando según la frecuencia de cada celda en la matriz. Es una de las librerías mas usadas a la hora de realizar matrices de confusión en proyectos de *machine learning*, por eso no se dudó en elegirla.

psutil



Figura 4.15: Logotipo de psutil [29]

librería en Python para la monitorización del sistema. Permite obtener información sobre el uso de la CPU, memoria, discos, etc. En nuestro proyecto lo usamos en el tiempo real de **Streamlit** para medir el rendimiento y los recursos consumidos por la inferencia en tiempo real y poder compararlo.

Cada una de estas librerías aportó al proyecto funcionalidades claves, desde la captura y preprocesamiento de imágenes, pasando por la construcción y entrenamiento del modelo CNN, hasta la implementación de la interfaz de usuario y la optimización en entornos de bajos recursos. Gracias a todo este ecosistema Python y estas herramientas, fue posible desarrollar el sistema al completo de una manera modular y eficiente consiguiendo gran parte de los objetivos marcados en el apartado dos de esta memoria. Todas las librerías mencionadas son de código abierto en su respectiva documentación y soporte de la comunidad en diversos foros, lo cual facilitó a la hora de desarrollar el proyecto. Además integrar estas librerías al entorno de la Raspberry Pi (ARM) como en mis propios ordenadores personales utilizados para desarrollar el proyecto (x86) debido a que al realizar `pip install` el gestor detecta nuestra plataforma y descarga automáticamente el `wheel` (formato de distribución empaquetada) adecuado garantizando así su compatibilidad.

Comparativa breve de alternativas como *PyTorch*, *PyTorch Mobile*... y su justificación

Es cierto que existen mas alternativas que no solo TensorFlow/Keras, algunas de ellas son muchos mas sencillas de implementar y bastante mas legibles al integrarse en el código. Un ejemplo de ello es **PyTorch**, también conocido generalmente como **Torch**, usada actualmente por la comunidad investigadora debido a que muchos desarrolladores consideran muy intuitivo y flexible a la hora de experimentar modelos. De hecho, generalmente se suele resumir esta comparativa como: **PyTorch** es útil para prototipos e

investigación y TensorFlow/Keras para proyectos a nivel de producción e implementaciones masivas [3].

Principalmente, se eligió TensorFlow/Keras debido a que uno de los primeros proyectos que se experimentó sobre entrenamiento y construcción de modelos CNN, fue una clasificación entre dos clases (perros y gatos) donde se utilizaba TensorFlow/Keras. Una vez haberlo experimentado por mi mismo, **la curva de aprendizaje resultó ser muy accesible** gracias a Keras al proveer abstracciones de alto nivel como por ejemplo *model.compile()* (configura el modelo, le dice como aprender) o *model.fit()* (entrena el modelo) que simplifican mucho más el entrenamiento. Además, contamos con gran cantidad de documentación junto a soporte comunitario para resolver dudas. Esto último fue muy importante en la comprensión de esta biblioteca: tutoriales, foros y ejemplo abundantes que fueron muy útiles y dieron confianza suficiente para continuar con el proyecto.

Por otro lado, la integración de TensorFlow Lite inclinó la balanza a favor de nuestro objetivo de introducir el modelo en la Raspberry Pi. Es cierto que PyTorch también cuenta con su propia alternativa optimizada llamada PyTorch Mobile, pero es cierto que la versión optimizada de TensorFlow suele ser **bastante mas rápida y liviana** gracias a su mayor optimización. Además, los tamaños binarios de TensorFlow Lite suelen ser mas pequeños que los binarios de PyTorch Mobile [22].

En resumen, aunque PyTorch ofrece una gran flexibilidad y popularidad entre los investigadores, la elección de TensorFlow/Keras estuvo respaldada en todo momento gracias a la facilidad que aporta su API Keras, su amplia documentación y comunidad existente y su optimización a la hora de funcionar en dispositivos sencillos como pueden ser micro-ordenadores accesibles y funcionales que encajan perfectamente con nuestros objetivos.

4.3. Herramientas de entrenamiento de modelos inicialmente utilizadas

Google Teachable Machine



Figura 4.16: Logotipo de Teachable Machine [31]

Teachable Machine es una herramienta web que posibilita crear modelos de aprendizaje profundo de manera sencilla y accesible para todos los usuarios[20].

En los inicios del proyecto, se exploró el uso de *Google Teachable Machine* como un **modelo a seguir** a la hora de realizar el proyecto y fijarme así en los distintos hiperparámetros que permitía editar y como estos afectaban directamente al entrenamiento. Esta web nos permite además de poder entrenar modelos de imagen estándar a color (224x224px), también nos permite entrenar proyectos de audio con pistas de un segundo de duración además de poder entrenar proyecto de posturas. Para nuestro proyecto, el indicado es el primero.

Dentro de de esta interfaz web podemos agregar cuantas clases necesitamos y cargar imágenes desde comprimidos hasta imágenes crudas. Una vez cargado todo nuestro *dataset* y comprobar que nuestras clases esta correctamente escritas, procedemos a entrenar pudiendo editar el número de épocas, el tamaño del lote (*Batch size*) o la tasa de aprendizaje (*learning rate* (*lr*)). Una vez entrenado (los entrenamientos tardan debido a que se entrena localmente en tu computadora), se puede exportar tanto a **TensorFlow**, **TensorFlow Lite** o **TF.js**.

Este modelo se utilizó previamente, pero se dejó de usar en el proyecto debido a la similitud de nuestro propio modelo entrenado localmente con nuestra propia red neuronal diseñada específicamente para nuestro conjunto de datos.

Creo que es totalmente conveniente haberla introducido en este apartado, debido a que nos ha permitido ver si era viable realizar un modelo con tantas clases y que, a la hora de realizar la inferencia, este clasificase correctamente, además de servir de inspiración para poder realizar el entrenamiento en **Streamlit** además de **usar sus modelos optimizados dentro de Streamlit** en vez de nuestros modelos optimizados debido

a que pesan muchísimo menos además de correr mejor en estos dispositivos computacionalmente limitados.

4.4. *Hardware* empleado

A lo largo del proyecto, se utilizaron diversos dispositivos *Hardware* tanto para desarrollo como para el entrenamiento y la ejecución final del sistema. A continuación se describen:

Portátil ASUS ZenBook 13

The logo for the ASUS ZenBook series, featuring the word "ASUS" in a smaller, sans-serif font followed by "ZenBook" in a larger, bold, sans-serif font.

Figura 4.17: Logotipo de ASUS ZenBook [40]

Se trata del **computador personal** usado sobre todo para desarrollar el proyecto, tanto para el desarrollo de la estructura y código, como para realizar la documentación. Cuenta con:

- **Sistema Operativo:** Windows 10 Pro 64 bits.
- **Procesador:** AMD Ryzen 7 5700U de 8 núcleos y 16 hilos, a una frecuencia base de 1,8GHz, x86.
- **Memoria:** 16GB (2x8GB) DDR4 en *Dual Channel* a 3733MHz CL36.
- **Tarjeta Gráfica:** AMD Radeon RX Vega 8 512MB VRAM (gráficos integrados).

Mac Pro 6.1 (finales del 2013)

The logo for the Mac Pro, featuring the words "Mac Pro" in a large, bold, sans-serif font.

Figura 4.18: Logotipo de Mac Pro [34]

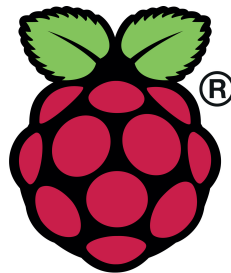
Conocida coloquialmente como "Trashcan", se trata del segundo computador personal de sobremesa pensado **principalmente para entrenar el**

modelo en su tarjeta gráfica aprovechando su memoria y su compatibilidad con macOS **metal** para acelerar el entrenamiento del modelo. Aunque tiene 2 GPU dedicadas, la versión de metal compatible con este Mac no permite entrenar el modelo con dos tarjetas gráficas a la vez, por lo tanto en entrenamiento solo se pudo acelerar sobre una de ellas.

Para poner en contexto, Esta tarjeta gráfica es similar a una NVIDIA GTX 960, con la diferencia de que la GPU que usa este Mac Pro tiene 2GB más de VRAM que aprovechará nuestro modelo.

- **Sistema Operativo:** macOS Monterey (macOS 12).
- **Procesador:** Intel Xeon E5-2697 V2 de 12 núcleos y 24 hilos, a una frecuencia base de 2,7GHz, x86.
- **Memoria:** 64GB (4x16GB) DDR3 ECC en *Quad Channel* a 1866MHz CL13.
- **Tarjeta Gráfica:** Dual AMD D700 12GB VRAM GDDR5 (2x6GB).

Raspberry Pi 5 + cámara (PS3 Eye)



Raspberry Pi

Figura 4.19: Logotipo de Raspberry Pi [39]

Es el dispositivo objetivo donde el proyecto según los objetivos, va a correr finalmente. Supone un salto de rendimiento notable respecto a la generación Pi 4, según el documento oficial de Raspberry Pi 5 sobre sus especificaciones, llega a triplicar el rendimiento que su anterior generación [18]. La Raspberry cuenta con puertos de alta velocidad USB3 donde conectaremos la cámara para proceder a correr **Streamlit** y empezar el reconocimiento.

- **Sistema Operativo:** Raspberry Pi OS 64 bits.

- **Procesador:** Broadcom BCM2712 de 4 núcleos y 4 hilos a una frecuencia base de 2.4GHz, ARM.
- **Memoria:** 8GB LPDDR4X.
- **Tarjeta Gráfica:** VideoCore VII.
- **Cámara:** PS3 Eye 60FPS 320x240.

En general, el *hardware* empleado fue suficiente para poder cumplir la mayoría de los objetivos del proyecto. La raspberry demostró ser capaz de manejar el proceso requerido gracias a su mejora con respecto a su anterior generación. En capítulos posteriores veremos las mediciones exactas, pero puedo adelantar que se logró un rendimiento en torno a los ~ 10 FPS con los respectivos modelos previamente optimizados con **TensorFlow Lite**.

5. Aspectos relevantes del desarrollo del proyecto

En este apartado se describen los **detalles mas destacables e importantes de la implementación practica del proyecto**. Se cubre desde la organización del código hasta las distintas fases de desarrollo, análisis, diseño, construcción del **dataset**, entrenamiento del modelo y la integración a la Raspberry Pi ayudado por **Streamlit** para crear un entrono amigable para el usuario.

5.1. Fase de análisis y diseño

Antes de lanzarnos a codificar, se realizó una fase de análisis para definir los objetivos. Se identificaron los requisitos funcionales (reconocer letras en tiempo real, interfaz simplificada, etc.) junto a los no funcionales (tiempo real, tasa de acierto elevado, etc). Estos puntos análisis se consideraron al inicio del proyecto:

- Para la parte de visión por computador, la duda era: ¿usar **detección de mano** o no?. La alternativa hubiera sido una red que directamente clasificara la letra desde la imagen cruda, sin procesado y completa (con mano y fondo sin especificar). Esto simplifica el *pipeline* (conjunto de pasos desde que se recogen hasta llegar al entrenamiento), pero requeriría un conjunto de datos bastante mayor y además la red tendría que aprender a ignorar el fondo y otros detalles lo que aumentarían la cantidad de épocas y datos para converger. Finalmente **se decidió incluir una etapa de detección de mano (con cvzone)**, por la robustez que aporta, como ya se explicó en la sección [3.2](#).

- En cuanto al **modelo de IA** a emplear, se decidió desde un inicio diseñar un modelo desde cero. Al decidir esto, tenemos absoluto control sobre la creación del modelo para poder adaptarlo a nuestro **dataset**. Pero es cierto que no encontramos los resultados esperados en esa red y optamos por cambiar su arquitectura implementando un **modelo de clasificación de imágenes preentrenado** llamado **EficientNetB0** con nuestro propio *dataset* el cual ayudó a obtener muy buenos resultados.
- Otro aspecto de diseño fue la construcción del *dataset*. Se tuvo que planificar como **obtener muchas imágenes en un intervalo corto de tiempo** y se decidió diseñar un fichero Python que se fijase solo en la mano gracias a la librería **cvzone**, establecer un tope de imágenes generadas para cada letra y mantener pulsado para tomar capturas rápidamente formando así un conjunto de datos sólido. Se pudo haber implementado varios conjuntos de datos extraídos de **Kaggle**, pero el reto era crear un *dataset* propio además de **utilizar los segmentos y nodos de la mano** gracias a la librería previamente mencionada, debido a que posteriormente se iba a clasificar con esa misma con nuestro clasificador.
- Se definió también la interacción de la aplicación, sobre si hacer una propia aplicación para que corriese **nativamente en la Raspberry** y así poder realizar una comparación efectiva o realizar una web que también corriese en el propio dispositivo y que en un futuro, pudiese ser alojada en un dominio. Se decidió por esto último además de realizarlo con la herramienta de **Streamlit**, en vez de usar herramientas como por ejemplo *Flask* debido a que es muy **fácil de implementar** además de requerir muy poco código para el propio *frontend* además debido al tiempo y a la sencillez de implementar la interfaz web, **Streamlit** fue la mejor opción para poder adaptarse al usuario de una forma sencilla y eficaz.
- También fue importante en el diseño prever la comparativa de rendimiento: ¿qué **métricas** íbamos a medir?:
 1. **Precisión de clasificación en validación**
 2. **Matriz de confusión**
 3. **Velocidad de inferencia (fps)** en Raspberry Pi con y sin optimizaciones.
 4. Uso de CPU + RAM durante la inferencia para ver la carga de los dispositivos

5. Y se deseaba medir la diferencia de CPU vs NPU si la *AI Pi camera* hubises funcionado. Al no ser posible convertir mi red neuronal convolucional, se propuso como idea a futuro.

En resumen, la fase de análisis y diseño sirvió para trazar un camino a seguir. Aún así muchos detalles se ajustaron dentro de la implementación, como es natural. Pero gracias al buen planteamiento inicial, pudimos completar la gran mayoría de objetivos.

5.2. Construcción del *dataset*

Para la construcción del conjunto de datos, se necesita **recoger un gran volumen de datos** debido a que cuanto mayor sea este, más datos tiene el modelo para aprender y poder fijarse en detalles y patrones de las imágenes. A continuación se detalla la metodología y el mecanismo de forma teórica:

Definición de clases (alfabeto ASL)

las clases a reconocer son las 26 letras del alfabeto ingles en su representación en ASL. cabe reiterar que en este alfabeto:

- Las letras A-I, K-J se realizan con la mano estática cada una con su respectivo gesto
- Las letras J y Z dinámicas.
 1. La letra J se realiza haciendo una J en el aire con el dedo meñique. Al ser un letra con **movimiento** (y nuestro modelo no detecta ese movimiento, si no solamente imágenes estáticas). Por lo tanto, la idea fué **recoger ese gesto una vez finalizado**, es decir, una vez acabado el recorrido de dibujar la J con el dedo meñique apuntando hacia abajo:

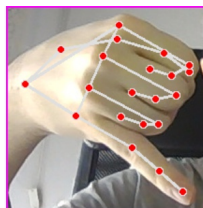


Figura 5.1: Imagen letra J extraida del conjunto de datos

2. Al Igual que la letra J, la letra Z se traza moviendo el dedo índice en forma de Z, muy parecido a la letra D, pero con el dedo pulgar recogido para así poder diferenciarlas:

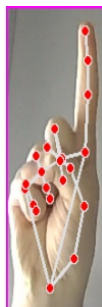


Figura 5.2: Imagen letra Z extraída del conjunto de datos

Así es como se estableció el *dataset*, En un principio estuvo compuesto por 500 imágenes por cada clase dando un total de: $500 \times 26 = 13000$ imágenes, pero mas adelante, para tener un conjunto de datos **ligeramente mas variado**, mi hermana completó el *dataset* voluntariamente añadiendo 500 imágenes por cada letra más otorgándonos un total de: $(500 \times 2) \times 26 = 26000$ imágenes totales **con procesado** las cuales, a día de hoy, componen el conjunto de datos.

Captura y preprocesado de imágenes

Realizar fotografías es una labor tediosa, sobre todo cuando requiere completar un *dataset* de 26000 imágenes. Además de fotografiar todo el conjunto de datos, también es necesario procesar cada muestra para **garantizar su calidad** para el posterior entrenamiento del modelo.

Captura semiautomática ("*burst mode*")

Al mantener pulsada la tecla de captura (por defecto la "s" por *save*), el programa entra en **modo ráfaga** y toma automáticamente varias imágenes. Un contador detiene la ráfaga al llegar al número máximo de muestras preestablecidas, de modo que todas las letras acaben con el mismo número de ejemplos. Así realizamos una recogida de datos mucho más amena y rápida.

Detección y recorte de la región de interés

Se utiliza el detector de manos de *cvzone* para localizar en tiempo real la mano y obtener su cuadro de detección. El detector de manos nos da un **rectángulo** (x, y, w, h) que encierran nuestra mano. Para poder recortar esa Zona con un pequeño margen y sin salirnos de la imagen, calculamos las esquinas del recorte, asegurándonos de no usar coordenadas fuera del fotograma:

$$x_1 = \max(0, x - \text{OFFSET}) \text{ (borde izquierdo)}$$

$$y_1 = \max(0, y - \text{OFFSET}) \text{ (borde superior)}$$

$$x_2 = \min(W_{\text{img}}, x + w + \text{OFFSET}) \text{ (borde derecho)}$$

$$y_2 = \min(H_{\text{img}}, y + h + \text{OFFSET}) \text{ (borde inferior)}$$

De este modo definimos la sub-imagen recortada $I_{\text{crop}} = I[y_1 : y_2, x_1 : x_2]$. Ese recorte se escala y centra dentro de un **lienzo blanco** de tamaño 300×300 , ajustando su ancho y su alto de la relación de aspecto (h/w) de la mano. Para ello

1. Calculamos la relación de aspecto de la mano

$$\text{aspectRatio} = \frac{h}{w}$$

- Si $\text{aspectRatio} > 1$, la mano es **mas alta que ancha**.
- Si $\text{aspectRatio} \leq 1$, la mano es **mas ancha que alta**.

2. Redimensionamiento

- Caso alto:
 - Factor de escala $k = \frac{300}{h}$
 - Nuevo ancho $w' = \lceil k \cdot w \rceil$
 - Redimensionamos el recorte a $(w', 300)$
- Caso ancho:
 - Factor de escala $k = \frac{300}{w}$
 - Nuevo ancho $h' = \lceil k \cdot h \rceil$
 - Redimensionamos el recorte a $(300, h')$

3. Cálculo de márgenes

Para centrar el recorte dentro de este "lienzo blanco":

- Si redimensionamos por alto, el margen horizontal es:

$$w_{\text{gap}} = \left\lceil \frac{300 - w'}{2} \right\rceil$$

y pegamos la imagen redimensionada en:

$$I_{\text{white}}[:, w_{\text{gap}} : w_{\text{gap}} + w'] = I_{\text{res}}$$

- Si redimensionamos por ancho, el margen vertical es:

$$w_{\text{gap}} = \left\lceil \frac{300 - h'}{2} \right\rceil$$

y pegamos en:

$$I_{\text{white}}[h_{\text{gap}} : h_{\text{gap}} + h', :] = I_{\text{res}}$$

De este modo cada mano utiliza totalmente uno de los ejes del lienzo y queda **perfectamente centrada** en la imagen 300×300 sin tener que reducir el tamaño por captura debido a que utilizamos márgenes dinámicos en vez de estáticos.

Descarte con imagen en blanco

Si durante el guardado de imágenes el detector no encuentra la mano (ya sea por mala iluminación, desenfoque o por que se salió de la perspectiva de la cámara), en lugar de descartar la iteración se **guarda una imagen completamente blanca** de tamaño 300×300 . Esto mantiene constante el número de muestras por clase además de enseñar al modelo a distinguir entre ausencia de mano y gestos válidos.

5.3. Implementación y entrenamiento del modelo

En esta sección se verán aspectos relevantes a la construcción del modelo, desde sus dificultades hasta su respectiva versión final.

Carga, procesamiento y *split* de datos

Todo el conjunto de datos se carga en el entrenamiento del modelo gracias a la herramienta de *Keras* llamada `image set from directory`, donde lee todas las carpetas (una por clase) y divide **el 80 % de esas imágenes para datos de entrenamiento** y reserva el otro **20 % para datos de validación**.

Con el 80 % disponemos de suficientes ejemplos para ajustar los pesos de la red y capturar características o patrones presentes en los datos.

Cada imagen viene siempre asociada como tupla con su etiqueta, formando una tupla (imagen, etiqueta) que se mantiene durante todo el **pipeline**, nunca se separan imágenes de sus etiquetas

Data augmentation

Lo que realiza este bloque es alterar cada lote de imágenes al vuelo (mejor conocido como *on the fly*) para que el modelo vea **mas variedad de imágenes** (y más si nuestro conjunto de datos es relativamente pequeño) y no memorice así patrones fijos.

Guardamos los datos en cache preprocesados en memoria para acelerar pasadas posteriores y con *prefetch* mientras la GPU esta ocupada entrenando el mini-lote actual de imágenes, **TensorFlow** usa por detrás la CPU libre para leer y procesar en segundo plano el segundo lote de datos para que la GPU cuando termina de procesar el **batch**, ya tiene el **batch + 1** siguiente para procesar de inmediato sin quedarse esperando a que se carguen. Esta funcionalidad es básicamente para que se **adelante trabajo minimizando tiempos muertos** entre la carga de esos mini-batch en GPU.

Modelo Principal

El modelo central empleado finalmente es una CNN de clasificación que aprovecha *transfer learning* con **EfficientNetB0** previamente entrenada en **ImageNet** de *Keras*. Para ello, se instancia **EfficientNetB0** con `include_top = False` (excluyendo sus capas finales de clasificación) y `weights = imagenet`, de modo que **solo se utiliza la parte convolucional para extraer las características de nuestro conjunto de datos**. Además, se congela esta base estableciendo `trainable=False`, lo que evita que sus pesos se actualicen durante el entrenamiento en el nuevo conjunto de datos, aunque es cierto que se desactivó por que el programa fallaba al pasar a la base.

la arquitectura completa se construye de forma secuencial y consiste en los siguientes bloques:

Layer (type)	Output Shape	Param #
lambda (Lambda)	(None, 224, 224, 3)	0
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4049571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
batch_normalization (Batch Normalization)	(None, 1280)	5120
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 26)	33306

Figura 5.3: Imagen de la arquitectura de la red neuronal

- Preprocesamiento de entrada (*Lambda*): La capa es la capa de entrada que **prepara las imágenes** del *dataset* **en formato esperado** por EfficientNet ([0–255] cada pixel de la imagen) para que las entiendan.
- Base EfficientNet-B0 (congelada): La red EfficientNet-B0 principalmente procesa la imagen y **extrae distintos mapas de características** que aportan información. Prácticamente es un bloque gigantesco de capas convolucionales y activaciones (ReLU, etc.) que **ya "sabe"** extraer bordes, texturas y formas de manos.
- GlobalAveragePooling2D: Tras la base convolucional preentrenada se aplica un *pooling* global promedio. Esta capa promedia cada capa de características especialmente convirtiendo la salida de (altura x anchura x canales, en este caso (7x7x1280)) en un **vector de 2 dimensiones** cuyo tamaño es igual al numero de canales de salida. Así se llevan las características a un vector que resume la información mas relevante.

- BatchNormalization: Es una capa que se utiliza para aumentar la estabilidad de entrenamiento. Básicamente, **normaliza las activaciones del vector características** para que tengan media 0 y varianza 1 dentro de cada *mini-batch* y así conseguir que el entrenamiento sea más estable y rápido.
- Dropout: Esta capa durante el entrenamiento **“apaga” aleatoriamente el 20 % (en nuestro caso) de las neuronas** en esa capa, es decir, en cada *mini-batch* en esa capa apaga al azar el 20 % de esos valores, en este caso 1280 neuronas, forzando al modelo a no depender siempre de los mismos canales, mejorando así la generalización y reduciendo el sobreajuste.
- Capa densa final con Softmax: la última capa es **totalmente conectada** con tantas neuronas como clases tiene nuestro proyecto (toma las 1280 valores como entrada y produce 26 salidas), de la A a la Z. Usa la función de activación *softmax* que **convierte cada valor en una probabilidad entre 0 y 1** asegurando que todas ellas sumen 1. Además incorpora regularización L2 en los pesos, **penalizando valores muy grandes** y reduciendo así el riesgo de memorizar ejemplos de entrenamiento.

El modelo se compila usando el optimizador **Adam** (uno de los mas comunes dentro del entrenamiento de redes neuronales) con una tasa de aprendizaje (*learning rate*) relativamente normal (0.0003 en nuestro caso) que permiten ajustar con cuidado los pesos de la cabeza sin destrozar los filtros preentrenados de **EfficientNet**. Como función de perdida usamos **sparse_categorical_crossentropy**, el cual es ideal cuando las etiquetas vienen como enteros (en nuestro caso de 0 que sería la letra A hasta el 25 que representaría la letra Z). Además incluimos la métrica **accuracy** (precisión) para **monitorizar en cada época** el porcentaje de aciertos en entrenamiento y validación (entre 1-0). Con esta configuración, el modelo ya está listo para ejecutar el bucle del entrenamiento con **model.fit**, donde **Adam** actualizará los pesos minimizando la perdida y las otras capas ayudarán a mejorar la robustez del entrenamiento.

Callbacks

En este modelo hemos introducido dos Callbacks para controlar el entrenamiento otorgándoles un colchón de 70 épocas.

- **EarlyStopping**: **Detiene el entrenamiento** si la variable que esta monitorizando, en nuestro caso el *validation accuracy* **no mejora** según la paciencia que tenga, (en nuestro caso 10 épocas). Si no mejora en esas épocas preestablecidas, **restaura los mejores pesos** que hubo en la mejor época con mejor rendimiento de validación, asegurando así no quedarnos con un estado peor por sobre-entrenar.
- **ReduceLROnPlateau**: Reduce el **learning rate** a la mitad si la validación no mejora según la paciencia establecida, en nuestro caso 3 épocas hasta un mínimo de 0.000001 (1e-6). Esto permite **ajustes finos** cuando el modelo se atasca y deje de converger para que el entrenamiento siga convergiendo aunque sea lentamente.

Entrenamiento

En cada época el modelo recorre todo el conjunto de datos de entrenamiento en **lotes** o **batches** preestablecidos con su respectivo tamaño (dependiendo la ram o vram que tengamos en nuestra computadora, podemos aumentar o disminuirlo). con los datos de validación comprueba como aprende el modelo además de indicarle los **callbacks** para que los aplique en el entrenamiento.

Guardado + Evaluación y Gráficas

El modelo se exporta con `model.save` que guarda la arquitectura + pesos en un fichero de Keras `.h5`, se pudo haber guardado como `.keras`, pero al utilizar en fases iniciales la herramienta **Teachable Machine**, esta exportaba sus modelos con la extensión `.h5`, por lo tanto, también realizamos ese guardado con ese mismo formato.

Las gráficas que se han representado son 3 en total:

- **Curvas de pérdida (*loss*) y precisión (*accuracy*)** tanto de la parte de entrenamiento como de validación. Estas gráficas nos dan indicios de sobreajustes indicándonos en que épocas se producen.
- **Matriz de confusión**: una tabla de 26x26 (una fila y columna por cada letra A-Z), las **filas** representan las **clases reales** (`y_true`) y las **columnas** las **predicciones del modelo** (`y_pred`). Una diagonal mucho mas resaltada significa que el modelo acierta la letra correcta con frecuencia. Es cierto que algunos valores se pueden encontrar fuera de la diagonal, si estos valores son algo destacados, el modelo puede

estar confundiendo letras. Esto nos ayuda a identificar que pares de clases son mas difíciles de distinguir entre sí (por ejemplo entre la M y la N (muy similares entre sí)).

Random Search

Para poder ajustar los hiperplarmetros del modleo base se utilizó la herramienta de Keras llamada **KerasTuner** con las estrategia **RandomSearch**. Esto permitiío explorar diferentes combinaciones de hiperparámetros clave y encontrar el conjutno mas efectivo, se defivineron como variables a optimizar:

- **Dropout rate:** se buscó entre los parámetros: 0.2, 0.3 y 0.4.
- **Learning rate:** se probó entre los parámetros típicos para Adam: 0.001 (1e-3), 0.0003 (3e-4), 0.0001(1e-4). Se añade 0.0003 y no 0.0005 porque es un parámetro muy reconocido y habitual en Adam.
- **L2 regularization:** se exploró entre los parámetros 0.001 (1e-3), 0.0005 (5e-4) y 0.0001 (1e-4).

Estos rangos se han escogido porque, por un lado se cubren los valores más usados para redes convolucionales preentrenadas, y por otro permiten a RandomSearch encontrar de manera eficiente sin muchos intentos (*trials*). Hemos elegido 30 *trials* porque, aunque el espacio de búsqueda sea reducido con 27 ($3 \times 3 \times 3 = 27$) combinaciones posibles, el random seach pero con 27 ensayos exactos habría riesgo de poder repetir alguna configuración y dejar otra sin probar debido a que **RandomSearch** no prueba configuraciones distintas, si no que son aleatorias, por lo que si que es posible que dos *trials* acaben usando exactamente los mismos valores debido a que no hay ningún mecanismo interno que evite repeticiones. Por ello, con 30 *trials* aumentamos la probabilidad de cubrir casi todas las combinaciones sin disparar el coste computacional. Además, añadimos 7 épocas a cada trial con una paciencia de parada temprana de 3 para descartar los distintos intentos que causen sobreajuste.

Finalmente, una vez acabado el **RandomSearch**, los mejores hiperparámetros que otorgó en un principio dejaba de aprender después de la quinta época, por lo tanto, cogimos los mejores segundos hiperparámetros, donde solo se diferenciaba el learning rate, bajando de 0.001 a 0.0003:

- **Dropout rate:** 0.2 .

- Learning rate: 0.0003 (3e-4).
- L2 regularization: 0.0001 (1e-4).

Guardado del modelo y conversión a *TFLite*

En el proyecto se utiliza un modelo entrenado en *Keras* que se almacena en un archivo *.h5* (punto flotante de 32 bits, FP32) el cual para poder exportarlo a la Raspberry, se debe convertir antes a un archivo *TensorFlow Lite*, un formato ligero y optimizado **para la inferencia en dispositivos embebidos**.

Para ello, utilizamos un *script* que se ubica en nuestra carpeta *utils*, el cual carga nuestro modelo entrenado *.h5*, lo transforma a un archivo *.tflite* con precisión FP32 (igual que nuestro modelo sin optimizado, pero eliminando todo lo innecesario para el entrenamiento) y luego aplica "*quantization*".^a media precisión FP16 (16 bits) para generar un segundo *.tflite* para poder **comparar el desempeño** de cada uno en Raspberry, resultados que expondremos próximamente. Esta cuantización como podemos observar reduce a la mitad el tamaño de los pesos, que a menudo acelera la inferencia pero por el contrario perdemos ligeramente exactitud.

Probé ambos modelos optimizados, tanto de *Teachable Machine* como los propios, y después de compararlos observamos que nuestro modelo optimizado **no era lo suficientemente ligero** ni esta lo suficientemente optimizado para correr en Raspberry Pi 5, por lo que, se usaron los extraídos de *Teachable machine* los cuales tienen un mejor rendimiento en tiempo real: 10 FPS vs 3 FPS. Una gran mejora a futuro a considerar.

5.4. Aplicación Streamlit (clasificador por imagen, tiempo real y módulo de entrenamiento)



Figura 5.4: Logotipo del proyecto desarrollado

Para poder representar toda la funcionalidad finalmente se decidió usar este *framework* basado en Python pensado para aplicaciones web de *machine learning* y ciencia de datos [23]. En este apartado, describimos como se estructuró la aplicación, sus funcionalidades y la forma en que se integra el entrenamiento con el propio conjunto de datos.

Nuestra aplicación se dividió conceptualmente en cuatro secciones principales: **Inicio**, **Clasificación en tiempo real**, **clasificación por imagen estática** y **Entrenamiento interactivo**. Es cierto que en todo momento se muestra una barra lateral inicialmente desplegada donde podemos navegar por nuestra aplicación además de elegir a través de un desplegable que modelo preferimos (FP32 o FP16) para las respectivas comparaciones a la hora de realizar la inferencia. A continuación se detalla cada sección con imágenes de la interfaz:

Inicio

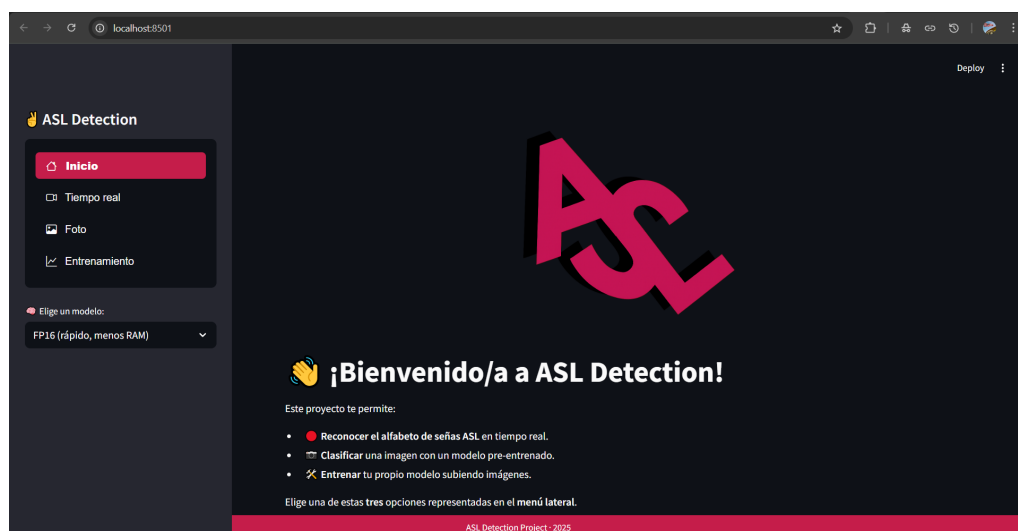


Figura 5.5: imagen del inicio del proyecto

Es nuestra página *index* donde se describen las 3 distintas funcionalidades de la aplicación además de mostrar el logotipo en grande para diferenciar nuestra aplicación dando un toque personal. Después de la breve descripción, te anima a navegar a través del menú insertado en la barra lateral.

Clasificación en tiempo real

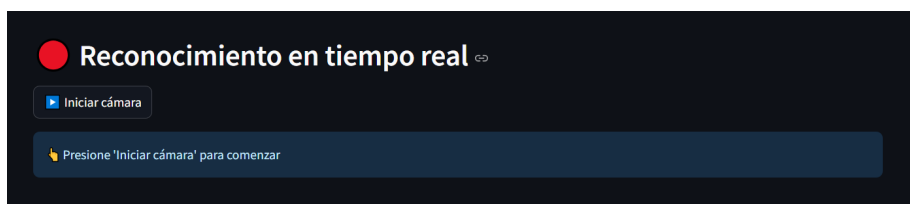


Figura 5.6: Captura de la interfaz inicial de reconocimiento en tiempo real

Esta es la funcionalidad central. Al entrar en esta sección, únicamente se encuentra un botón **“Iniciar Cámara”** esperando a que activemos la cámara con una breve descripción señalando al botón **“Clasificación en tiempo real”**. Si no detecta ninguna cámara conectada al equipo nos mostrará **“error al mostrar la cámara”** de lo contrario, aparecerá un mensaje de carga **“Iniciando cámara, por favor espere”**. Una vez realizado, se empezará a ver vídeo proveniente de la cámara **esperando recibir unas manos para realizar la inferencia** y así predecir el gesto correspondiente al alfabeto

ASL junto la muestra de fotogramas por segundo en la esquina superior derecha de la pantalla. Además, justo debajo de la imagen tenemos una gráfica que se actualiza en tiempo real midiendo el uso de la CPU y RAM además de los Fotogramas por segundo de la inferencia para compararlos mas adelante. Si nos fijamos, donde se ubicaba el botón de “**Iniciar Cámara**” habrá cambiado totalmente a “**Detener cámara**” . Debo aclarar que, bajo el capó, se crea una instancia de la clase *RealTimeASLClassifier* (proveniente del script orientado a la clasificación en raspberry pi en la carpeta *classifier*, configurada para usar el modelo entrenado (en Raspberry Pi, se optó por cargar el modelo *TFLite* en lugar del *.h5* por eficiencia, adaptando ligeramente la clase *classifier.py* para usar el intérprete *TFLite*).

clasificación por imagen estática

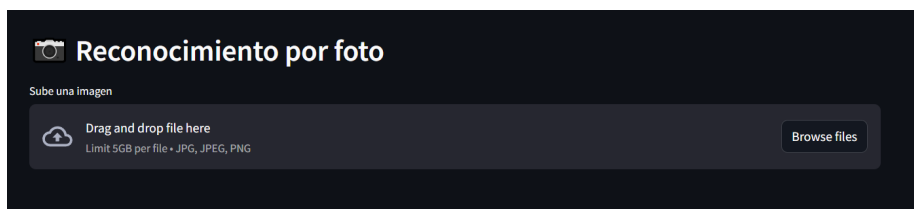


Figura 5.7: Captura de la interfaz de reconocimiento por foto

En esta sección, el usuario puede subir una foto desde el explorador de archivos gracias al botón “**Browse files**” o arrastrar la foto hacia el recuadro gracias al *drag and drop* solo con formatos de imágenes válidos (*.jpg*, *.jpeg*, *.png*). Después de subir la foto, internamente se lee y se decodifica la imagen leyendo los bytes, se decodifica a BGR que es el formato usado por *OpenCV*. Después, internamente a la hora de clasificar hay una instancia de la clase *classify_image* (proveniente de *classifier_photo.py*) la cual clasifica la foto y recibe una etiqueta y una confianza en porcentaje, convierte la imagen de BGR a RGB y la muestra. Además, mientras se está realizando la predicción de esa imagen estática, se dibuja una barra de progreso como indicador de actividad. Como funcionalidad extra, puedes descargar la imagen con el nombre de *uploaded.png*.

Entrenamiento interactivo

Esta es una característica extra que se incluyó debido a la herramienta utilizada en los inicios del proyecto *Teachable Machine* que nos inspiró para poder realizar esta funcionalidad extra en nuestra web.

Al igual que en la herramienta realizada por Google, el usuario puede decidir hasta 26 clases, la vigesimosexta incluida, donde puede introducir su propio conjunto de datos separados por clases sin límite de imágenes por clase (en un futuro se podría llegar a limitar el número de imágenes a añadir por clase si tenemos pensado alojar nuestra web en internet). Una vez el usuario ya ha introducido todo el *dataset* que desee, puede editar tres hiperparámetros, los cuales son:

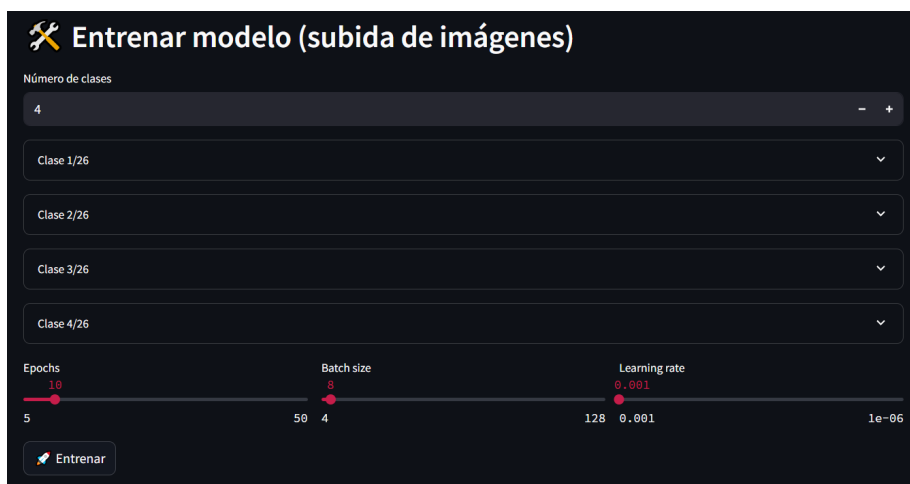


Figura 5.8: Captura de la interfaz de entrenamiento

- **Cantidad de épocas (*Epochs*)**: número de veces que el modelo recorrerá todo el conjunto de entrenamiento. Se establece un límite de 5 a 50 con un *slider* el cual aumenta 1 a 1.
- **Tamaño del lote (*Batch size*)**: numero de muestras que procesa el modelo en cada paso antes de actualizar los pesos. Se establece un límite de 4 a 128 donde el usuario puede decidir dependiendo de la cantidad de memoria que disponga su computadora a través de un *slider* que aumenta de 4 en 4.
- **Tasa de aprendizaje (*learning rate*)**: la magnitud de los ajustes que se aplican a los pesos en cada iteración de optimización. Se establece un limite de 0.001 a 0.000001 el cual el usuario deberá elegir un *learning rate* que considere adecuado a través del *slider*, por defecto está definido a 0.001.

Una vez realizado, se entrena donde el usuario podrá ver en una grafica en tiempo real la precisión de, tanto el conjunto de validación como del

conjunto de entrenamiento además de dibujar una barra de progreso para que se le mantenga informado al usuario en todo momento de su entrenamiento. Una vez finalizado, el usuario podrá descargar un archivo comprimido el cual contendrá el modelo exportado en *.h5* y en *.tflite (FP32)* además de un fichero de texto donde se almacenarán las diferentes clases.

Gía de estilo y paleta visual

En la interfaz de usuario, hemos empleado una paleta inspirada en los colores oficiales de Raspberry Pi Foundation como por ejemplo el rojo vibrante característico de la marca [13]. Este color se usa principalmente en botones, el *footer* además de implementar un fondo oscuro jugando con tonalidades grisáceas claras y oscuras para no representar una interfaz visual monótona.

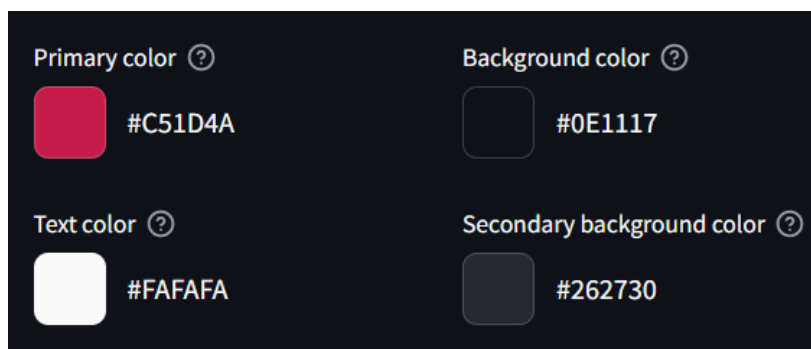


Figura 5.9: Colores utilizados en el proyecto junto a su valor hexadecimal

5.5. Integración en Raspberry Pi 5

Para desplegar la aplicación en Raspberry Pi 5, primero convertimos el modelo entrenado a un formato ligero y optimizado con *TensorFlow Lite* que permita una inferencia fluida en un sistema con bajos recursos. Una vez incluida únicamente la visión optimizada del modelo en la interfaz web, procedemos a configurar el dispositivo

1. Instalación de Python ARM: se instala la misma versión de Python compilada para la arquitectura ARM de las Raspberry Pi 5
2. Bajamos el [repositorio del proyecto](#) que se encuentra público en GitHub.

3. Instalamos todas las dependencias manual o automáticamente usando ejecutando el *script* en **bash** proporcionado en la raíz del proyecto.
4. Iniciar la web con el mismo archivo **bash** que instala dependencias (idempotente si previamente ya han sido instaladas).

Se ha decidido introducir los requerimientos en un archivo **bash** para automatizar el proceso y hacer un poco mas simple el proceso de instalación de dependencias. Además, este archivo inicializa automáticamente un entorno virtual, de modo que las librerías de otros proyectos no se vean afectadas.

5.6. Incompatibilidad de mi red neuronal con *Pi AI Camera*

Al comenzar el proyecto, como uno de los objetivos principales planteamos ejecutar la inferencia en nuestra Raspberry Pi aprovechando la nueva cámara AI lanzada en colaboración con Sony. Esta cámara integra un sensor Sony IMX500 de 12 MP, diseñado para acelerar redes neuronales convolucionales. Sin embargo, no pudimos utilizarla porque nuestro modelo no pudo exportarse al formato cuantizado que la cámara requiere, teniendo que redirigir ese objetivo a realizar la inferencia con una cámara normal sin aceleración y convirtiendo nuestro modelo a uno mas optimizado gracias a *TensorFlow Lite*

Estos son los pasos se tuvieron que realizar para poder convertir nuestro modelo al formato compatible con la cámara IA.

- Modelo FP32: debimos entrenar primero nuestra red como es lógico, pero luego debimos añadir la extensión **.keras** que es 100 % compatible con nuestra extensión **.h5**
- Cuantización int8: debimos volver a entrenar simulando int8 para que al red aprendiese a manejar baja precisión junto al uso de *strip-quantization* con el cual eliminamos todos los bloques de simulación y quedarnos con pesos ya en int8
- IMX500 Converter: Instalamos esta librería debido a que es la herramienta oficial de Sony que compila ese **.keras int8** limpio en binario optimizado, aquí es donde no funcionaba la conversión. la herramienta simplemente se quedaba ejecutándose infinitamente.

Finalmente, al no ver progresión en esta idea, se decidió hibernar e implementarla en un futuro para no perder mucho tiempo a la hora de alcanzar nuestros objetivos.

5.7. Pruebas y comparativas

En este apartado se verán las distintas pruebas de entrenamiento entre modelos y comparativas previamente realizadas en la Raspberry Pi 5 junto a una breve descripción y explicación de las gráficas insertadas

Fotogramas por segundo (FPS)

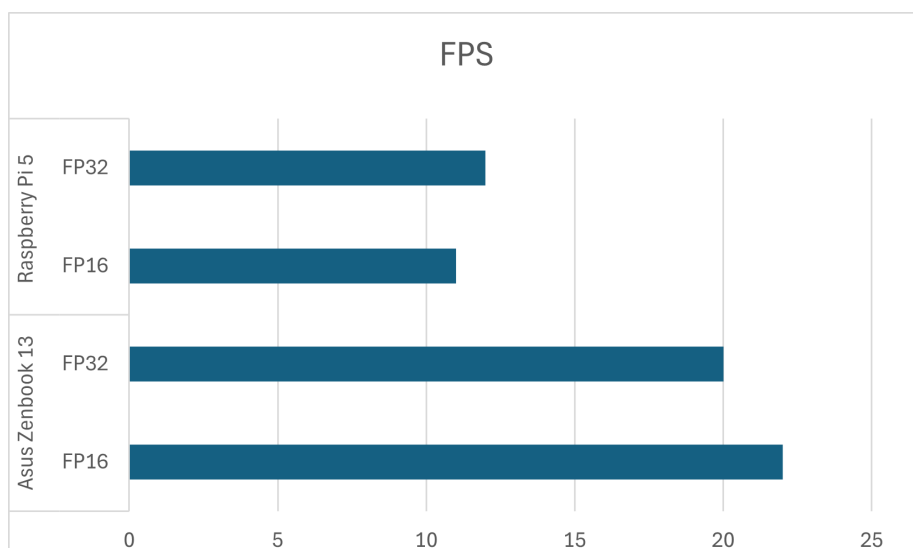


Figura 5.10: Comparativa de FPS realizada entre un portátil y la Raspberry Pi 5

Gráficas del modelo (Pérdida (*Loss*) y Exactitud (*Accuracy*))

Pérdida (*Loss*)

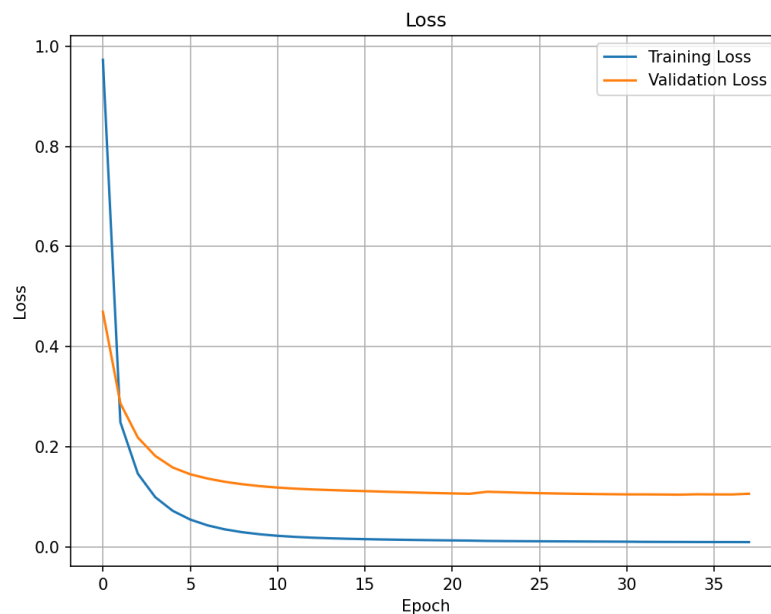


Figura 5.11: Gráfica sobre la pérdida durante el entrenamiento del modelo

La gráfica muestra como la función de pérdida a demás de empezar con una perdida ya relativamente muy baja, desciende rápido durante las primeras épocas: Al inicio se puede observar que la pérdida de entrenamiento cae de ~ 1 a $\sim 0,1$ en las primeras 5 épocas, mientras que la validación cae de $\sim 0,5$ a $\sim 0,17$. a partir de ahí el *Loss* sigue aproximándose a cero, pero la de validación podemos observar que se establece en $\sim 0,1$. Esto indica que puede haber un **pequeño sobreajuste** al alargarse tanto el entrenamiento.

Exactitud (*Accuracy*)

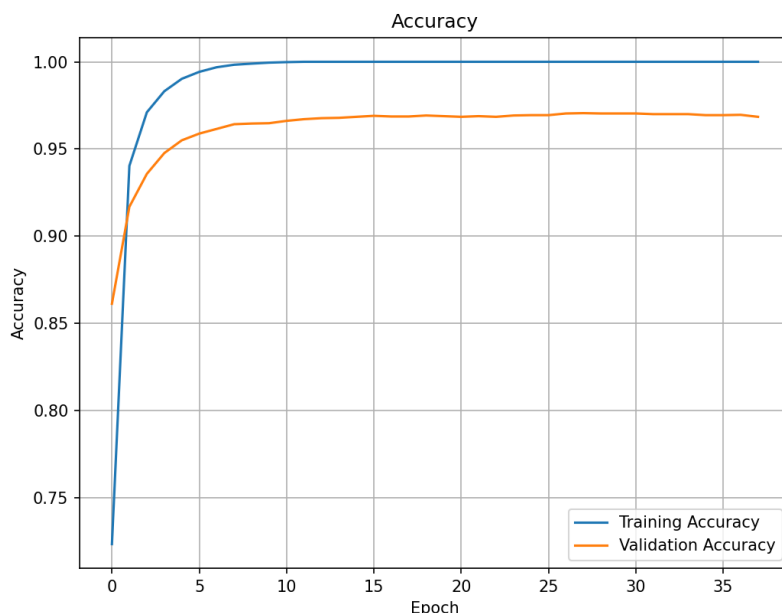


Figura 5.12: Gráficas sobre la exactitud durante el entrenamiento del modelo

Como podemos observar, es prácticamente la misma gráfica pero como si se le hubiese hecho un *flip* horizontal. La precisión del entrenamiento sube rápidamente de un $\sim 0,73$ a un $\sim 0,99$ en las primeras 5 épocas, mientras que la precisión de la validación alcanza el $\sim 0,96$ pero, al igual que en la pérdida, la validación se establece y no sigue subiendo alcanzando el **97,06 % en validación**, confirmando ese pequeño sobreajuste que nos da la idea de que el modelo aunque es cierto que ha capturado con mucho éxito la mayoría de patrones generales, hay exceso de entrenamiento que no aporta mejoras adicionales.

Además, es cierto que gracias a nuestros ajustes de **Early Stopping** (acortando el entrenamiento) y **ReduceLROnPlateau** (rebajando la tasa de aprendizaje) evitamos que el modelo caiga en un sobreajuste extremo.

Matriz de confusión

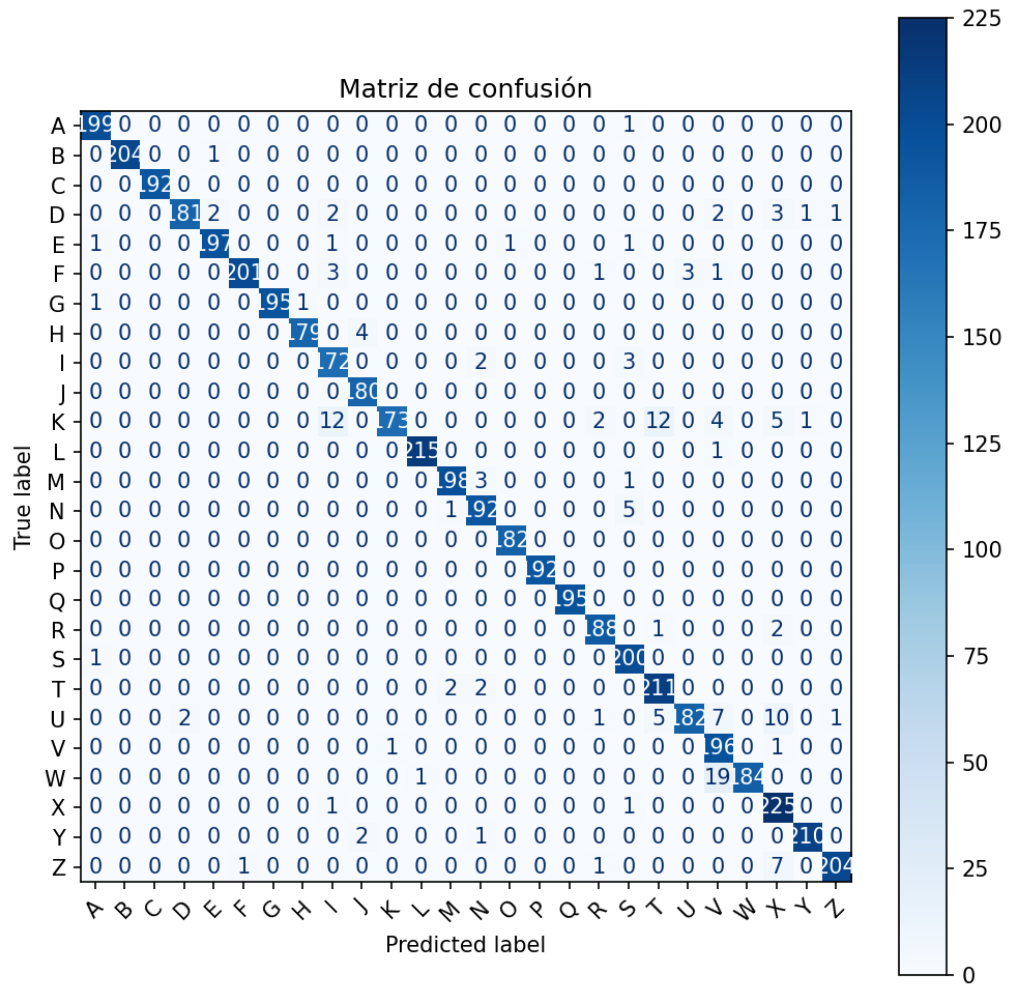


Figura 5.13: Matriz de confusion del modelo

Todo ese bloque toma únicamente muestras de validación (el 20 % reservado), extrae sus etiquetas (*labels*) y las predicciones del modelo, y a partir de ahí construye la matriz de confusión. De esta forma se comprueba cómo de bien generaliza la red a datos que el modelo nunca vio durante el entrenamiento. Es cierto que En algunas letras como W y V el modelo se puede llegar a confundir, sobre todo porque estos gestos son sumamente parecidos entre sí, al igual que K y T. Pero, podemos asegurar que el entrenamiento fue todo un éxito viendo como resultado la diagonal resaltada.

6. Trabajos relacionados

En este campo hay numerosos estudios sobre el reconocimiento de lenguaje de signos. Muchos de ellos de estos proyectos y emplean técnicas de visión por computador diversas. Buscar este tipo de proyectos nos ha ayudado a recoger características para nuestra propia aplicación las cuales serán útiles para el correcto desarrollo del proyecto.

En esta sección van a revisar 4 proyectos/aplicaciones relevantes y se comparan sus enfoques y objetivos con los objetivos principales de nuestro proyecto.

6.1. Reconocimiento de gestos de Google AI Edge

Google lanzó varias soluciones sobre diferentes modelos como una API de inferencia de LLM, detección de objetos, segmentación de imágenes, entre otras. La más interesante para comparar con este proyecto es la solución de [Gesture Recognizer de MediaPipe](#) (reconocimiento de gestos) la cual pre-entrenada junto con [MediaPipe Hands](#), puede reconocer ciertas señas comunes, aunque sin ningún tipo de alfabeto, si no gestos como: puño cerrado, pulgar hacia arriba, etc., además de detectar ambas manos. Uno de los apartados destacables de este proyecto es que es multiplataforma además de poder correr en el navegador. La diferencia notable con nuestro proyecto es que solo reconoce unas pocas clases, mas en concreto 7 clases donde se pueden utilizar ambas manos a la vez. Es cierto que esta web ha motivado a implementar un modelo que pueda correr en cualquier dispositivo (mas optimizado) sin requerir de un hardware potente a nuestra disposición. Por eso decidimos cuantizar nuestro modelo de [TFLite](#) a [FP16](#) [9].

6.2. Sign Language Recognition with Convolutional Neural Networks

Este proyecto es el mas similar que he encontrado con respecto al realizado. Estos alumnos de la universidad de Standford desarrollaron un sistema de reconocimiento del alfabeto ASL utilizando una red neuronal convolucional, entrenado en un conjunto de datos masivo de imágenes (más de 200k imágenes de 29 clases, incluyendo: espacio, nada y borrar) [4]. Una vez entrenado, reportaron una precisión de prácticamente el 100 % en la clasificación de las letras, mas concretamente un 99.31 % que, comparándolo con su conjunto de datos de mas de 200k imágenes, es un porcentaje altísimo y fiable en la clasificación del alfabeto ASL [4]. Sin embargo, su implementación estaba orientada a entornos de escritorio que requiriesen un hardware decente que no consideraba restricciones de tiempo real en hardware limitado.

6.3. Google Teachable Machine

Es cierto que en el pasado utilizamos esta herramienta para poder avanzar en el proyecto con un modelo fiable. Además, esta herramienta diseñada por Google ha servido de inspiración para poder realizar la funcionalidad de entreno en nuestra web desarrollada con **Streamlit** junto a los respectivos hiperparámetros editables, característica también añadida en nuestra interfaz.

6.4. SignALL

SignAll es una empresa que desarrolló un sistema complejo con múltiples cámaras y sensores que captan profundidad para traducir signos a voz y viceversa. También es capaz de reconocer vocabulario amplio y no solo limitarse al alfabeto además de poder reconocer gestos en movimiento, pero requiere un *set-up* muy específico con 3 cámaras diferentes (una cámara captura el movimiento del cuerpo, otra cámara capta el movimiento de las manos y otra expresiones faciales) [7].

Esta es una solución extrema y con un nivel muy superior a nuestro proyecto, pero esta característica ha servido para motivarnos a utilizar un modelo que se pueda ejecutar con cualquier cámara existente sin tener que invertir en un *set-up* siendo así mucho mas accesible.

7. Conclusiones y líneas de trabajo futuras

En este último capítulo se presentan las conclusiones generales del proyecto y se sugieren líneas de trabajo a futuro para posteriores actualizaciones del proyecto.

7.1. Conclusiones

Desde el punto de vista técnico y teórico, este proyecto deja varias conclusiones que merecen la pena comentar:

- El proyecto me ha resultado bastante entretenido, pero sobre todo útil debido a que, gracias a su realización, he obtenido conocimientos de diversas áreas, como por ejemplo: técnicas de visión de computador, entrenamiento y diseño de redes neuronales convolucionales y la integración de múltiples componentes tanto *software* (todas las librerías, entornos, etc.) como *hardware* (uso de Raspberry Pi) que tanto empeño invertí para integrarlo al proyecto. Todas las librerías y documentaciones referenciadas me sirvieron para poder formarme y obtener conocimientos nuevos sobre el campo del *deep learning*.
- Es cierto que los objetivos se lograron, demostrar que es posible reconocer el lenguaje de signos ASL (en alfabeto) mediante inteligencia artificial en un sistema computacionalmente limitado como es la Raspberry Pi 5, con un buen desempeño y de manera interactiva. Esta demostración valida esa hipótesis inicial sobre la capacidad de las

CNN de reconocer gestos y características muy concretas y poderlas llevarlas a cabo en sistemas limitados.

- La experiencia que me ha dado este proyecto sobre como gestionar el tiempo y sobre todo tomar decisiones cruciales ha sido de gran valor para poderlo aplicar a mi día a día. Cuando un camino no funciona, barajamos nuevas alternativas, el caso mas notable fue la cámara AI de Raspberry con NPU debido a que invertimos mucho tiempo en investigar su uso además de investigar como poder convertir nuestro modelo al formato correcto. Debido a la poca documentación al ser relativamente nueva y a la poca información que nos otorgaba el conversor, decidimos redirigir ese objetivo de forma satisfactoria. Adaptar las estrategias a los recursos disponibles fue crucial para concluir el proyecto exitosamente.

7.2. Líneas de trabajo futuro

Este proyecto debe tener una serie de actualizaciones para poder convertirlo en algo mas cercano a un producto o herramienta universal. Para poder llegar a ello, hay que implementar nuevas funcionalidades que pueden ser añadidas de forma satisfactoria en líneas de trabajo futuras.

Los siguientes puntos son una serie de sugerencias sobre líneas de trabajo que pueden aplicarse en el futuro:

- **Aplicar modelos nuevos en nuestra aplicación:** Se sugiere un apartado nuevo para poder subir el modelo que se ha entrenado en el apartado de entrenamiento ya sea optimizado o no y poder elegirlo para clasificarlo en tiempo real o de forma estática subiendo una captura o foto.
- **Modelo CNN propio optimizado:** En el futuro podría diseñarse un modelo CNN que además de obtener buenos resultados en el entrenamiento, también sea optimizado para poder trasladarlo a la Raspberry Pi sin depender de herramientas externas como **Teachable Machine**.
- **Implementar cámara AI de Raspberry:** Adaptar el modelo de forma que sea compatible para utilizar el conversor **IMXConverter** de Sony para poder utilizar la cámara IA de Raspberry Pi y así obtener mejor rendimiento en la inferencia con Raspberry.

- **Implementar Google Collab:** Otra mejora a futuro sería adaptar el código de entrenamiento del modelo a **Jupyter Notebook** para usar el aceleramiento gráfico que otorga **Google Collab** y así realizar el entrenamiento de una forma mucho más rápida que ejecutar el entrenamiento en local.
- **Alojamiento web con Streamlit Community Cloud:** Se propone alojar la web en internet para que múltiples usuarios puedan acceder a ella de forma remota y sin instalar dependencias sin que de problemas la clasificación en tiempo real con **Streamlit Community Cloud** u otra herramienta de *hosting*.
- **Internacionalización:** Una vez alojado, Internacionalizar la web insertando nuevos idiomas para que usuarios de todo el mundo puedan utilizarla.
- **Uso de lengua escrita como apoyo:** Aumentar las clases del conjunto de datos para añadir soluciones como: borrar, espacio, coma, punto, etc. para poder aplicar un lienzo de escritura dentro de la propia web.

Bibliografía

- [1] Google AI. Marcadores de mano proporcionados por mediapipe hands. URL <https://ai.google.dev/static/edge/mediapipe/images/solutions/hand-landmarks.png?hl=es-419>.
- [2] Google AI. Guía de detección de puntos de referencia en la mano, 2025. URL https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker. [Internet; consultado 9-junio-2025].
- [3] Farooq Alvi. Pytorch vs tensorflow in 2025: A comparative guide of ai frameworks, 2024. URL <https://opencv.org/blog/pytorch-vs-tensorflow/>. [Internet; consultado 12-junio-2025].
- [4] Anusha Kuppahally y Malavi Ravindran Arnav Gangal. Sign language recognition with convolutional neural networks. *Revista Vínculos*, pages 1–10, 2024. URL <https://cs231n.stanford.edu/2024/papers/sign-language-recognition-with-convolutional-neural-networks.pdf>. [Internet; consultado 21-junio-2025].
- [5] Language Services Associates. Why is there a shortage of certified american sign language interpreters?, 2024. URL <https://lsa.inc/why-is-there-a-shortage-of-certified-american-sign-language-interpreters/>. [Internet; consultado 7-junio-2025].
- [6] Wikimedia Commons. Logotipo de pycharm. URL https://commons.wikimedia.org/wiki/File:JetBrains_PyCharm_Product_Icon.svg.

- [7] CORDIS. El primer sistema que traduce lengua de signos automáticamente, 2019. URL <https://cordis.europa.eu/article/id/411590-first-system-to-automatically-translate-sign-language/es>. [Internet; consultado 21-junio-2025].
- [8] Francisco Javier Gallegos Funes Diana Alejandra, Contreras Alejo. Comparación de dos técnicas propuestas hs-cbcr y hs-ab para el modelado de color de piel en imágenes. *Research in Computing Science*, page 12, 2016. URL https://rsc.cic.ipn.mx/2016_114/Comparacion%20de%20dos%20tecnicas%20propuestas%20HS-CbCr%20y%20HS-ab%20para%20el%20modelado%20de%20color%20de%20piel.pdf. [Internet; consultado 11-junio-2025].
- [9] Google AI Edge. Guía de tareas de reconocimiento de gestos, 2025. URL https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer. [Internet; consultado 16-junio-2025].
- [10] Jeffrey Erickson. ¿qué es la inferencia de ia?, 2024. URL <https://www.oracle.com/es/artificial-intelligence/ai-inference/>. [Internet; consultado 14-junio-2025].
- [11] Universidad Europea. Qué son las redes neuronales convolucionales y para qué se utilizan, 2025. URL <https://universidadeuropea.com/blog/redes-neuronales-convolucionales/>. [Internet; consultado 12-junio-2025].
- [12] Python Software Foundation. Logotipo de python 11. URL <https://user-images.githubusercontent.com/11718525/197611877-583a0bb2-a8fb-4275-8827-39f2f06ade6c.png>.
- [13] Raspberry Pi Foundation. Colours, 2025. URL <https://bits.raspberrypi.org/#!/Colours>. [Internet; consultado 18-junio-2025].
- [14] Keras. Logotipo de keras. URL <https://keras.io/img/logo-k-keras-wb.png>.
- [15] Kwamikagami. Countries where american sign language, or a close derivative thereof, is the national language for the deaf, or where used alongside another sign language. URL https://en.m.wikipedia.org/wiki/File:ASL_map_%28world%29.png.
- [16] Luis Llamas. Logotipo de tensorflow/keras. URL <https://www.luisllamas.es/wp-content/uploads/2019/02/tensorflowkeras.png>.

- [17] Vizdx LLC. Logotipo de cvzone. URL <https://usercontent.one/wp/www.computervision.zone/wp-content/uploads/2021/04/Untitled-design-92.png>.
- [18] Raspberry Pi Ltd. Raspberry pi 5 product brief. *Raspberry Pi Datasheets*, page 2, 2025. URL <https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>. [Internet; consultado 9-junio-2025].
- [19] y D. A. Saa M. J. Flores; D. J. Robayo. Histograma del gradiente con múltiples orientaciones (hog-mo) detección de personas. *Revista Vínculos*, pages 138–144, 2015. URL <https://revistas.udistrital.edu.co/index.php/vinculos/article/view/10991/11830>. [Internet; consultado 11-junio-2025].
- [20] Teachable Machine. Google creative lab, 2019. URL <https://teachablemachine.withgoogle.com/faq>. [Internet; consultado 14-junio-2025].
- [21]Codigo Maquina. Mapas de características. URL https://www.youtube.com/watch?v=zBqnWfTwCgc&list=LL&index=5&ab_channel=CodigoMaquina.
- [22] Ayush Mishra. Tensorflow lite vs pytorch mobile for on-device machine learning, 2024. URL <https://www.analyticsvidhya.com/blog/2024/12/tensorflow-lite-vs-pytorch-mobile/>. [Internet; consultado 12-junio-2025].
- [23] DataCamp Nadia mhadhbi. Tutorial de python: Streamlit, 2024. URL <https://www.datacamp.com/es/tutorial/streamlit>. [Internet; consultado 15-junio-2025].
- [24] United Nations. Día internacional de las lenguas de señas, 2025. URL <https://www.un.org/es/observances/sign-languages-day>. [Internet; consultado 7-junio-2025].
- [25] National Institute on Deafness and Other Communication Disorders. American sign language, 2021. URL <https://www.nidcd.nih.gov/health/american-sign-language>. [Internet; consultado 11-junio-2025].
- [26] OpenCV.ai. Opencv ai, 2025. URL <https://www.opencv.ai/>. [Internet; consultado 12-junio-2025].

- [27] OpenCV.org. Introduction to opencv — opencv 4.10.0 documentation, 2024. URL <https://docs.opencv.org/4.10.0/d1/dfb/intro.html>. [Internet; consultado 12-junio-2025].
- [28] Samaya Madhavan Piyush Madan. Relación entre inteligencia artificial, aprendizaje automático y aprendizaje profundo. URL <https://developer.ibm.com/developer/default/articles/an-introduction-to-deep-learning/images/AI-ML-DL.png>.
- [29] Giampaolo Rodola. Logotipo de psutil. URL <https://repository-images.githubusercontent.com/20101515/b0ffffb6-b2fa-44c8-a81b-50ac40fa1077>.
- [30] TensorFlow.org. Keras: la api de alto nivel para tensorflow, 2024. URL <https://www.tensorflow.org/guide/keras?hl=es>. [Internet; consultado 12-junio-2025].
- [31] There's An AI For That. Logotipo de teachable machine. URL <https://media.theresanaiforthat.com/icons/teachable-machine.svg>.
- [32] Civil Rights Division U.S. Department of Justice. Ada requirements: Effective communication, 2020. URL <https://www.ada.gov/resources/effective-communication/>. [Internet; consultado 11-junio-2025].
- [33] Wikipedia. Logotipo de latex, . URL https://es.m.wikipedia.org/wiki/Archivo:LaTeX_logo.svg.
- [34] Wikipedia. Logotipo de mac pro, . URL https://es.wikipedia.org/wiki/Archivo:Mac_Pro.svg.
- [35] Wikipedia. Logotipo de matplotlib, . URL https://es.wikipedia.org/wiki/Archivo:Matplotlib_icon.svg.
- [36] Wikipedia. Logotipo de numpy, . URL https://en.m.wikipedia.org/wiki/File:NumPy_logo_2020.svg.
- [37] Wikipedia. Logotipo de opencv, . URL https://es.wikipedia.org/wiki/Archivo:OpenCV_Logo_with_text.png.
- [38] Wikipedia. Logotipo de overleaf, . URL https://commons.wikimedia.org/wiki/File:Overleaf_Logo.svg.
- [39] Wikipedia. Logotipo de raspberry pi, . URL https://www.esa.int/var/esa/storage/images/esa_multimedia/images/2016/10/raspberry_pi_logo/16166824-1-eng-GB/Raspberry_Pi_logo_pillars.jpg.

- [40] Wikipedia. Logotipo de asus zenbook, . URL https://es.wikipedia.org/wiki/Archivo:ASUS_ZenBook_logo.svg.
- [41] Wikipedia. Logotipo de tensorflow metal, . URL https://commons.wikimedia.org/wiki/File:Apple_Metal_logo,_version_2.svg.
- [42] Wikipedia. Logotipo de scikit-learn, . URL https://commons.wikimedia.org/wiki/File:Scikit_learn_logo_small.svg.